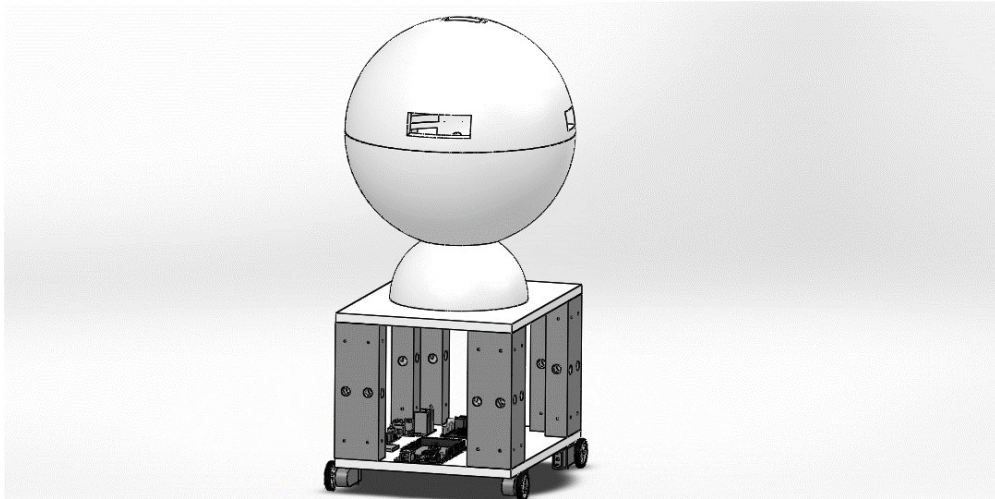


TERCEIRA EDIÇÃO REVISTA E AMPLIADA

Robotinics Rev. 3

Robótica, IoT e Inteligência Artificial com Arduino, Raspberry Pi e
TCHATGPT



Marcelo Maurin Martins

Terceira edição revista e ampliada • 2026

FICHA DA EDIÇÃO

Robotinics Rev. 3

Robótica, IoT e Inteligência Artificial com Arduino, Raspberry Pi e TCHATGPT

Autor e responsável pelo projeto: Marcelo Maurin Martins

Terceira edição revista e ampliada, 2026.

Idioma: português do Brasil.

Projeto	Snapshot editorial
Robotinics	39c1b4610f1b51883a2d9fcdbccba2c672dafad3
TCHATGPT	15d5e1a7780088701716896cfe9fb3afe0e7b71a

ATENÇÃO Projeto educacional e experimental. Baterias de íons de lítio, ferramentas, soldagem, motores e partes móveis exigem proteção, supervisão e componentes adequados. A IA nunca substitui as barreiras determinísticas de segurança.

Fontes do projeto: [repositório Robotinics](#) e [repositório TCHATGPT](#).

Nota editorial: o snapshot analisado do Robotinics não contém uma licença formal. A publicação pública desta edição deve ser acompanhada pela licença escolhida pelo autor.

À minha querida esposa, Daniela Machado.

Amor eterno.

Sumário

Clique em uma entrada para navegar no documento.

<u>Apresentação da terceira edição.....</u>	<u>7</u>
<u>O que foi corrigido.....</u>	<u>7</u>
<u>Como usar este livro.....</u>	<u>8</u>
<u>Convenções.....</u>	<u>9</u>
<u>Agradecimentos.....</u>	<u>10</u>
<u>Preparação do projeto - Arquitetura, segurança e método.....</u>	<u>11</u>
<u>1. O Robotinics como plataforma de aprendizagem.....</u>	<u>12</u>
<u>1.1 Robô, IoT e sistema ciberfísico.....</u>	<u>12</u>
<u>1.2 Objetivos da Rev. 3.....</u>	<u>12</u>
<u>1.3 Público-alvo.....</u>	<u>12</u>
<u>1.4 Organização do repositório.....</u>	<u>12</u>
<u>1.5 Estratégia de documentação.....</u>	<u>13</u>
<u>2. Arquitetura da terceira edição.....</u>	<u>14</u>
<u>2.1 Separação de responsabilidades.....</u>	<u>14</u>
<u>2.2 Caminho de um comando.....</u>	<u>14</u>
<u>2.3 Perfis de implantação.....</u>	<u>15</u>
<u>2.4 Estados operacionais.....</u>	<u>16</u>
<u>2.5 Interfaces.....</u>	<u>16</u>
<u>3. Segurança e método de trabalho.....</u>	<u>18</u>
<u>3.1 Hierarquia de segurança.....</u>	<u>18</u>
<u>3.2 Regras de bancada.....</u>	<u>18</u>
<u>3.3 Controle de mudanças.....</u>	<u>18</u>
<u>3.4 Critérios de conclusão por etapa.....</u>	<u>19</u>
<u>3.5 Segurança de credenciais e dados.....</u>	<u>19</u>
<u>3.6 Limitações declaradas.....</u>	<u>19</u>
<u>3.7 Caderno de construção.....</u>	<u>19</u>
<u>Parte I - Mecânica.....</u>	<u>21</u>
<u>4. Dimensionamento mecânico.....</u>	<u>22</u>
<u>4.1 Requisitos antes da escolha dos motores.....</u>	<u>22</u>
<u>4.2 Forças de movimento.....</u>	<u>22</u>
<u>4.3 Exemplo corrigido de tração.....</u>	<u>22</u>
<u>4.4 Aderência e limite de tração.....</u>	<u>23</u>
<u>4.5 Redução: trocar velocidade por torque.....</u>	<u>23</u>
<u>4.6 Torque dos braços.....</u>	<u>24</u>
<u>4.7 Centro de gravidade e estabilidade.....</u>	<u>25</u>
<u>5. Estrutura e montagem mecânica.....</u>	<u>26</u>
<u>5.1 Acervo mecânico.....</u>	<u>26</u>
<u>5.2 Material e impressão.....</u>	<u>26</u>
<u>5.3 Base.....</u>	<u>27</u>
<u>5.4 Corpo, braços e cabeça.....</u>	<u>28</u>
<u>5.5 Alternativas à impressão 3D.....</u>	<u>29</u>
<u>5.6 Critérios de aceitação mecânica.....</u>	<u>29</u>
<u>Parte II - Eletrônica.....</u>	<u>30</u>
<u>6. Alimentação, bateria e distribuição.....</u>	<u>31</u>
<u>6.1 Grandezas fundamentais.....</u>	<u>31</u>
<u>6.2 Conjunto 3S.....</u>	<u>31</u>
<u>6.3 Carga segura.....</u>	<u>31</u>
<u>6.4 BMS, fusível e chave geral.....</u>	<u>32</u>
<u>6.5 Trilhos separados.....</u>	<u>32</u>
<u>6.6 Conversores.....</u>	<u>33</u>
<u>6.7 Autonomia.....</u>	<u>33</u>
<u>6.8 Capacitores e ripple.....</u>	<u>33</u>
<u>6.9 Medição correta.....</u>	<u>33</u>
<u>6.10 Critérios de aceitação elétrica.....</u>	<u>34</u>
<u>7. Eletrônica, sensores e atuadores.....</u>	<u>35</u>
<u>7.1 Arduino e níveis lógicos.....</u>	<u>35</u>

7.2 Shield com conectores carenados e polarizados.....	35
7.3 Mapa de pinos derivado do firmware.....	38
7.4 Driver de motores.....	39
7.5 Relés.....	40
7.6 LEDs.....	40
7.7 Sensor ultrassônico.....	41
7.8 Servos.....	42
7.9 Sensores analógicos.....	42
7.10 Desacoplamento e compatibilidade eletromagnética.....	43
7.11 Checklist eletrônico.....	43
Parte III - Software e inteligência artificial.....	45
8. Firmware Robotinics Rev. 3.....	46
8.1 Preparar, compilar e gravar.....	46
8.2 Por que reorganizar.....	46
8.3 Módulos.....	47
8.4 Inicialização segura.....	47
8.5 Entrada serial limitada.....	47
8.6 Manual de comandos.....	48
8.7 Watchdog e heartbeat.....	48
8.8 Obstáculos e limites locais.....	48
8.9 Telemetria.....	48
8.10 Evitar fragmentação de memória.....	49
8.11 Testes do firmware.....	49
9. Raspberry Pi e serviços do robô.....	50
9.1 Papel do Raspberry Pi.....	50
9.2 Hardware atual recomendado.....	50
9.3 Instalação do sistema passo a passo.....	51
9.4 Perfil mínimo.....	52
9.5 Serviços com systemd.....	52
9.6 Dependências e instalador.....	53
9.7 Câmera e áudio.....	53
9.8 Armazenamento.....	53
9.9 Atualização e rollback.....	53
10. Comunicação, dados e segurança de software.....	55
10.1 Serial antes da rede.....	55
10.2 Servidores históricos.....	55
10.3 Portas e contratos.....	55
10.4 Interface de rede.....	55
10.5 Mensagens estruturadas.....	55
10.6 Banco de dados.....	56
10.7 Observabilidade.....	56
10.8 Testes de resiliência.....	56
Inteligência artificial aplicada ao robô.....	58
11. Preparando o TCHATGPT.....	59
11.1 O que é um LLM e o que ele faz no Robotinics.....	59
11.2 O projeto.....	60
11.3 Provedores.....	60
11.4 Configuração sem segredo no código.....	61
11.5 Chamadas assíncronas.....	61
11.6 Prompts e contratos.....	61
11.7 Autoteste.....	62
12. Agente de controle supervisionado.....	63
12.1 Componentes.....	63
12.2 Configuração segura.....	63
12.3 Registro de capacidade.....	63
12.4 Evento antes da ação.....	63
12.5 Confirmação compreensível.....	64
12.6 Descoberta determinística.....	64
12.7 Memória de sessão.....	65
12.8 Limites atuais.....	65
12.9 Testes adversariais.....	65
13. Voz e interação humana.....	66

13.1 Pipeline.....	66
13.2 Reconhecimento de fala.....	66
13.3 Voz não é autorização.....	66
13.4 Síntese tradicional e clonagem.....	66
13.5 Privacidade.....	67
13.6 Critérios de aceitação.....	67
14. Visão computacional.....	68
14.1 Correção histórica.....	68
14.2 Componentes atuais.....	68
14.3 Pipeline do Robotinics.....	69
14.4 Desempenho no Raspberry.....	70
14.5 Segurança e ética.....	70
14.6 Testes.....	70
15. Memória e RAG.....	72
15.1 Uso no robô.....	72
15.2 TAIRAG.....	72
15.3 Separação entre conhecimento e autoridade.....	72
15.4 Fontes e rastreabilidade.....	72
15.5 Fatiamento.....	73
15.6 Memória operacional.....	73
15.7 Avaliação.....	73
16. Comportamentos inteligentes e limites.....	74
16.1 Modos de operação.....	74
16.2 Habilidades apropriadas.....	74
16.3 Planejar, executar e verificar.....	74
16.4 Incerteza.....	75
16.5 Observabilidade de IA.....	75
16.6 Critério de promoção.....	75
Integração, validação e evolução.....	76
17. Sequência de construção.....	77
17.1 Etapa 0 - documentação e inventário.....	77
17.2 Etapa 1 - mecânica passiva.....	77
17.3 Etapa 2 - distribuição elétrica.....	77
17.4 Etapa 3 - Arduino sem atuadores.....	77
17.5 Etapa 4 - tração suspensa.....	78
17.6 Etapa 5 - servos e sensores.....	78
17.7 Etapa 6 - Raspberry e gateway.....	78
17.8 Etapa 7 - TCHATGPT em simulação.....	78
17.9 Etapa 8 - agente com hardware.....	78
17.10 Etapa 9 - voz, visão e RAG.....	78
17.11 Primeira operação no piso.....	79
18. Plano de testes e comissionamento.....	80
18.1 Pirâmide de testes.....	80
18.2 Matriz mínima.....	80
18.3 Medições elétricas.....	81
18.4 Medições mecânicas.....	81
18.5 Avaliação da IA.....	81
18.6 Injeção de falhas.....	82
18.7 Registro de ensaio.....	82
18.8 Critério de comissionamento.....	82
19. Manutenção e evolução.....	83
19.1 Inspeção periódica.....	83
19.2 Compatibilidade.....	83
19.3 Repositório reproduzível.....	83
19.4 Roadmap técnico.....	84
19.5 Publicação.....	84
Apêndice A - Lista de materiais orientativa.....	86
Apêndice B - Protocolo de comandos.....	87
B.1 Regras.....	87
B.2 Catálogo mínimo.....	87
B.3 Aliases legados.....	87
Apêndice C - Matriz de correções da edição anterior.....	88

<u>Apêndice D - Fórmulas de referência</u>	90
<u>Apêndice E - Matriz de plataforma e dependências</u>	91
<u>Apêndice F - Referências</u>	92
<u>Projetos</u>	10
<u>Documentação técnica oficial</u>	92
<u>Apêndice G - Glossário</u>	93
<u>Encerramento</u>	94

Apresentação da terceira edição

O Robotinics nasceu como um projeto multidisciplinar: um robô terrestre, aberto à experimentação, construído com peças impressas em 3D, eletrônica acessível, Arduino, Raspberry Pi e software livre. A primeira edição documentou uma jornada real de pesquisa e construção. Esta terceira edição recupera essa identidade: é um livro do tipo **faça você mesmo**, no qual um projeto já concluído é reconstruído diante do leitor, etapa por etapa, com os conceitos usados para tomar cada decisão.

A mudança principal não é apenas trocar versões de programas. A didática cronológica da edição original foi combinada às correções técnicas e à arquitetura segura da Rev. 3. O leitor primeiro constrói a mecânica, depois monta e mede a eletrônica e, por fim, instala o software. Em cada parte aparecem objetivo, conhecimentos necessários, materiais, procedimento, teste antes de avançar e resultado esperado. O Arduino continua responsável por tarefas determinísticas e de tempo real. O Raspberry Pi coordena serviços, sensores e comunicação. O TCHATGPT fornece modelos de linguagem, agentes, voz, visão, memória, RAG e observabilidade. Entre a decisão da IA e qualquer atuador existe uma camada obrigatória de validação e segurança.

Esta edição foi construída sobre duas referências congeladas:

Projeto	Referência editorial	Finalidade
Robotinics	commit 39c1b4610f1b51883a2d9fcdccba2c672dafa d3	Base da revisão editorial, mecânica, eletrônica, firmware e acervo histórico
TCHATGPT	commit 15d5e1a7780088701716896cfe9fb3afe0e7b7 1a	Componentes Lazarus/Free Pascal para IA e integração

O congelamento das referências não impede evolução. Ele apenas garante que o leitor consiga relacionar texto e código. Novas versões podem ser adotadas depois que as diferenças forem verificadas.

ATENÇÃO Este é um projeto educacional e experimental. Ele inclui motores, baterias de íons de lítio, ferramentas, soldagem e partes móveis. Não trabalhe energizado, não monte carregadores de lítio improvisados e não faça ligações à rede elétrica com base apenas neste livro. Use módulos certificados, proteção adequada e supervisão de pessoa qualificada.

O que foi corrigido

As correções desta edição incluem, entre outras:

- capacidade de células em série, tensão nominal e tensão máxima do conjunto 3S;
- processo de carga CC/CV, proteção e balanceamento de células de lítio;
- uso correto do multímetro para tensão e corrente;
- distinção entre força, torque, potência, energia e capacidade;
- cálculos de tração, aclave e torque dos braços;
- corrente de LED e dimensionamento do resistor limitador;
- definição de contatos normalmente aberto e normalmente fechado de relés;

- limites reais de conversores buck e separação das fontes de motores, servos e lógica;
- descrição do L298N, controle de velocidade por PWM e limitações do componente;
- frequência do HC-SR04, exemplos de código e requisitos de temporização;
- problemas de segurança e memória nos servidores C/C++ históricos;
- conflito de portas, comandos e arquivos ausentes no repositório atual;
- práticas antigas de Linux, banco de dados, PHP, OpenCV, voz e acesso remoto;
- navegação, referências, terminologia e organização editorial.

Uma matriz detalhada aparece no Apêndice C.

Como usar este livro

Após uma preparação curta sobre arquitetura, segurança e método, o projeto é dividido em **três partes**, como na proposta didática original:

1. **Mecânica:** dimensionar esforços e redução, fabricar as peças, montar base, corpo, braços, garra e cabeça, e verificar alinhamento e estabilidade.
2. **Eletrônica:** montar alimentação e proteções, instalar a shield com conectores polarizados, fabricar chicotes, ligar motores, servos e sensores e medir cada trilho.
3. **Software:** preparar Arduino e Raspberry Pi atuais, configurar comunicação, visão computacional, voz, LLMs, agentes e RAG e comissionar o sistema integrado.

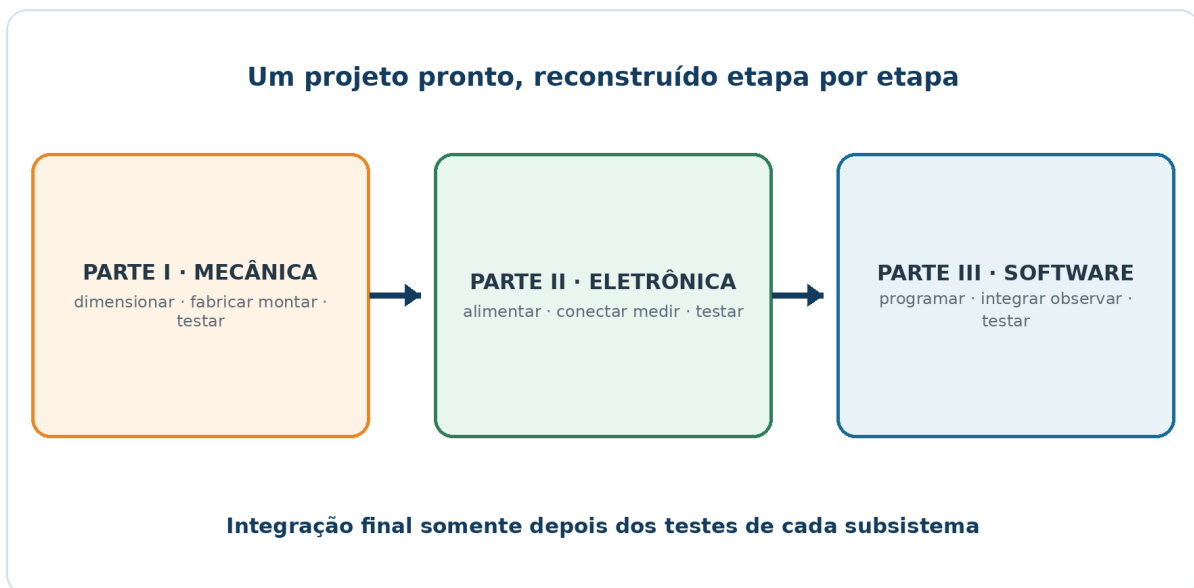


Figura 1 - Roteiro faça você mesmo: construir, testar e integrar as três partes do Robotinics.

Fonte: Elaboração própria.

O leitor não precisa instalar todas as ferramentas para montar o robô. Quem deseja apenas reproduzir o protótipo pode usar os arquivos prontos. Quem pretende alterar peças, placas ou software deve trabalhar com as fontes versionadas. Em ambos os casos, cada etapa termina com critérios de aceitação: não avance enquanto o subsistema atual não estiver estável.

Os capítulos de construção usam sempre a mesma sequência: **entender**, **separar**, **montar**, **inspecionar**, **testar** e **registrar**. Fotografias mostram componentes e montagens reais; diagramas elétricos e tabelas de pinos são a referência de ligação. Uma fotografia nunca substitui o esquema nem a ficha técnica.

Convenções

- Valores de tensão são indicados em volts (V), corrente em ampères (A), potência em watts (W), energia em watt-hora (Wh), força em newtons (N) e torque em newton-metro (N·m).
- Exemplos numéricos são didáticos. Quando um componente real tiver ficha técnica diferente, prevalece a ficha técnica.
- Comandos literais, nomes de arquivos e identificadores aparecem em fonte monoespaçada.
- Trechos marcados como **Experimental** não possuem validação suficiente na plataforma indicada.
- O termo **IA** engloba modelos de linguagem, visão, voz, classificação e agentes. Isso não significa autonomia irrestrita.

Agradecimentos

Esta obra resulta de anos de estudo, construção e colaboração. Permanecem os agradecimentos de Marcelo Maurin Martins a Deus, à família, aos amigos, aos professores do Centro Paula Souza, da ETEC e da FATEC, e às pessoas que revisaram ou apoiaram o projeto original. A memória do professor Danilo continua homenageada nesta edição.

Também são reconhecidas as comunidades de Arduino, Raspberry Pi, Free Pascal, Lazarus e software livre, cujo trabalho torna possível construir e compartilhar projetos como o Robotinics.

Preparação do projeto - Arquitetura, segurança e método

1. O Robotinics como plataforma de aprendizagem

1.1 Robô, IoT e sistema ciberfísico

Um robô é um sistema eletromecânico capaz de perceber o ambiente, processar informações e produzir ações. Um dispositivo de Internet das Coisas coleta ou recebe dados e os troca com outros sistemas. O Robotinics pertence aos dois grupos: possui sensores, atuadores, processamento local, comunicação em rede e serviços que podem ser integrados a aplicações externas.

A definição é útil porque evita reduzir o projeto a uma única placa. O comportamento emerge da cooperação entre estrutura mecânica, alimentação, eletrônica, firmware, sistema operacional e aplicações. Uma falha de energia pode parecer erro de software. Ruído de motor pode corromper uma leitura de sensor. Um atraso de rede pode tornar perigoso um comando de movimento. Por isso a unidade de projeto é o sistema completo.

1.2 Objetivos da Rev. 3

A terceira edição adota os seguintes objetivos verificáveis:

1. Manter o robô controlável mesmo quando a rede ou a IA estiver indisponível.
2. Impedir que texto livre seja convertido diretamente em sinal de motor.
3. Permitir descoberta determinística dos comandos publicados pelo firmware.
4. Separar energia de tração, servos e lógica.
5. Registrar versão, estado e falhas de cada subsistema.
6. Executar testes por etapas antes da montagem completa.
7. Oferecer modos local e remoto de IA sem obrigar o leitor a usar um provedor específico.
8. Preservar compatibilidade com o acervo mecânico e com comandos históricos quando isso não comprometer a segurança.

1.3 Público-alvo

O livro atende estudantes, professores, makers e desenvolvedores que desejam compreender a integração entre áreas. Não pressupõe experiência profissional em todas elas, mas exige cuidado. Um iniciante pode reproduzir as etapas de baixo risco; atividades envolvendo bateria, alta corrente, ferramentas ou alterações estruturais devem contar com orientação adequada.

Para programadores, o Robotinics oferece um caso real de sistemas embarcados, protocolos, Lazarus, Free Pascal, C/C++, Linux, visão, voz e IA. Para profissionais de eletrônica e mecânica, mostra como transformar componentes físicos em um sistema observável e programável.

1.4 Organização do repositório

No snapshot adotado, os arquivos principais encontram-se em:

Área	Caminho de referência	Conteúdo
Mecânica	<code>Mecanic/solidwork</code> e <code>Mecanic/stl</code>	Montagens, peças e arquivos para impressão 3D
Eletrônica	<code>Eletronic/eagle</code> , <code>Eletronic/pcb</code>	Placas, desenhos e arquivos históricos de fabricação
Arduino	<code>Software/arduino</code>	Firmware principal e firmware da cabeça
Raspberry	<code>Software/raspberry</code>	Serviços, câmera, voz e aplicações históricas
Banco e web	<code>Software/database</code> e <code>Software/site</code>	Scripts e interface histórica
Livro	<code>docs</code>	PDF da edição anterior

Algumas ferramentas Lazarus citadas na edição anterior foram removidas do branch principal em 2022. Elas permanecem recuperáveis no histórico, mas não devem ser apresentadas como se estivessem disponíveis no clone atual. A Rev. 3 substitui seu papel por exemplos baseados nos pacotes modulares do TCHATGPT.

1.5 Estratégia de documentação

Conceitos, cálculos, arquitetura e critérios de teste pertencem ao livro. Sequências de instalação que mudam rapidamente devem permanecer em documentação versionada no repositório. Essa divisão reduz obsolescência: o livro ensina por que e como validar; o repositório informa o comando exato compatível com determinada versão.

Antes de iniciar, registre em uma ficha de construção:

- revisão mecânica das peças;
- modelo das placas Arduino e Raspberry Pi;
- modelo, química e capacidade declarada da bateria;
- modelos dos motores e servos;
- versão do firmware;
- commit dos projetos Robotinics e TCHATGPT;
- sistema operacional e arquitetura;
- provedor ou servidor local de IA;
- resultados dos testes de bancada.

2. Arquitetura da terceira edição

2.1 Separação de responsabilidades

A arquitetura utiliza camadas. O Arduino executa controle de baixo nível: motores, servos, leituras simples, temporização, limites e parada. O Raspberry Pi executa serviços do robô: comunicação, captura de sensores de maior volume, supervisão e integração. O TCHATGPT interpreta intenção, organiza contexto e coordena componentes de IA. A interface humana apresenta solicitações e confirma ações sensíveis.

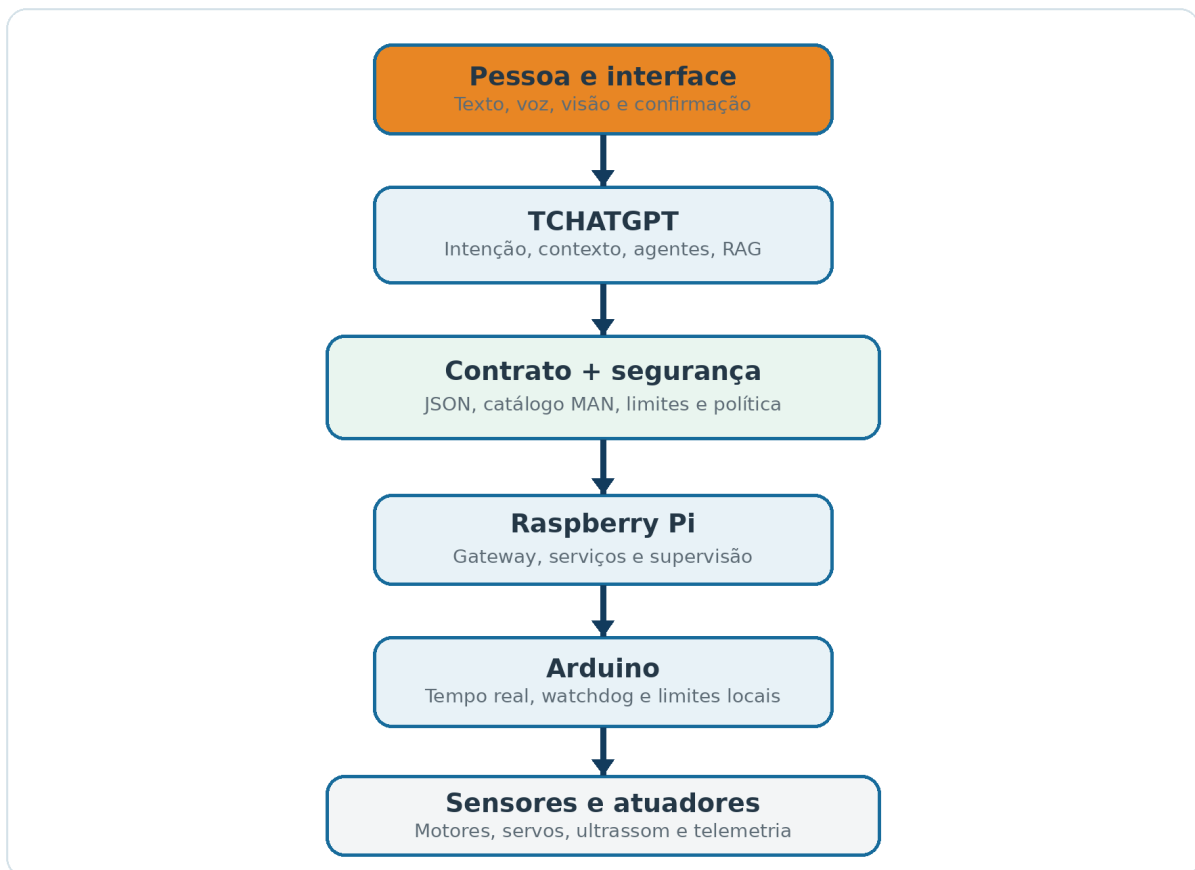


Figura 2 - Arquitetura em camadas da terceira edição.
Fonte: Elaboração própria.

Essa separação é deliberada. Modelos de linguagem são probabilísticos e podem produzir uma resposta incorreta. Motores exigem comportamento previsível. O modelo pode sugerir uma ação estruturada; somente o validador decide se a ação está no catálogo, se os parâmetros estão no intervalo e se o estado do robô permite executá-la.

2.2 Caminho de um comando

O caminho seguro de uma solicitação é:

1. O usuário fala ou digita uma intenção.
2. O sistema converte a entrada em texto e inclui apenas o contexto necessário.

3. O agente solicita ao modelo uma resposta em formato estruturado.
4. Um parser valida a estrutura; texto fora do contrato não é executado.
5. O comando é comparado ao catálogo descoberto por MAN.
6. A política de segurança valida estado, parâmetros, limites e necessidade de confirmação.
7. O usuário confirma ações físicas quando exigido.
8. O comando é enviado ao Arduino.
9. O firmware aplica seus próprios limites e watchdog.
10. A telemetria confirma o resultado; o agente informa o usuário.



*Figura 3 - Caminho supervisionado de um comando físico.
Fonte: Elaboração própria.*

O modelo nunca é a única barreira. Se ele inventar um comando, o catálogo bloqueia. Se usar um parâmetro fora da faixa, o validador bloqueia. Se a comunicação parar, o watchdog interrompe o movimento. Se um obstáculo for detectado, o firmware pode parar sem consultar a IA.

2.3 Perfis de implantação

Perfil A - controlador validado

O aplicativo Lazarus com TCHATGPT roda em Windows x64 ou Linux x64. O Raspberry Pi atua como gateway e computador de bordo. É o perfil recomendado para a primeira montagem porque essas plataformas possuem melhor evidência de suporte no projeto TCHATGPT.

Perfil B - IA embarcada no Raspberry Pi

O aplicativo Lazarus e os pacotes compatíveis rodam no Raspberry Pi ARM64. O modelo pode ser remoto ou local. No snapshot usado nesta edição, ARM64 está classificado como experimental; portanto, este perfil exige compilação, teste de dependências e validação de cada componente antes de controlar atuadores.

Perfil C - operação local sem nuvem

O TCHATGPT usa um endpoint OpenAI-compatible local, como llama.cpp ou neural-api. Esse perfil evita enviar dados do robô a um provedor externo, mas exige memória, armazenamento e capacidade computacional compatíveis com o modelo. Em hardware limitado, um modelo pequeno pode servir para classificação e comandos curtos, enquanto visão e voz usam componentes especializados.

2.4 Estados operacionais

O robô trabalha com quatro estados mínimos:

Estado	Movimento	Comandos de consulta	Transição permitida
DISARMED	Bloqueado	Permitidos	Para ARMED após verificação e confirmação
ARMED	Permitido dentro de limites	Permitidos	Para EXECUTING, DISARMED ou FAULT
EXECUTING	Somente ação corrente	Telemetria permitida	Retorna a ARMED ou vai a FAULT
FAULT	Bloqueado e saída segura	Diagnóstico permitido	Para DISARMED após correção e reset

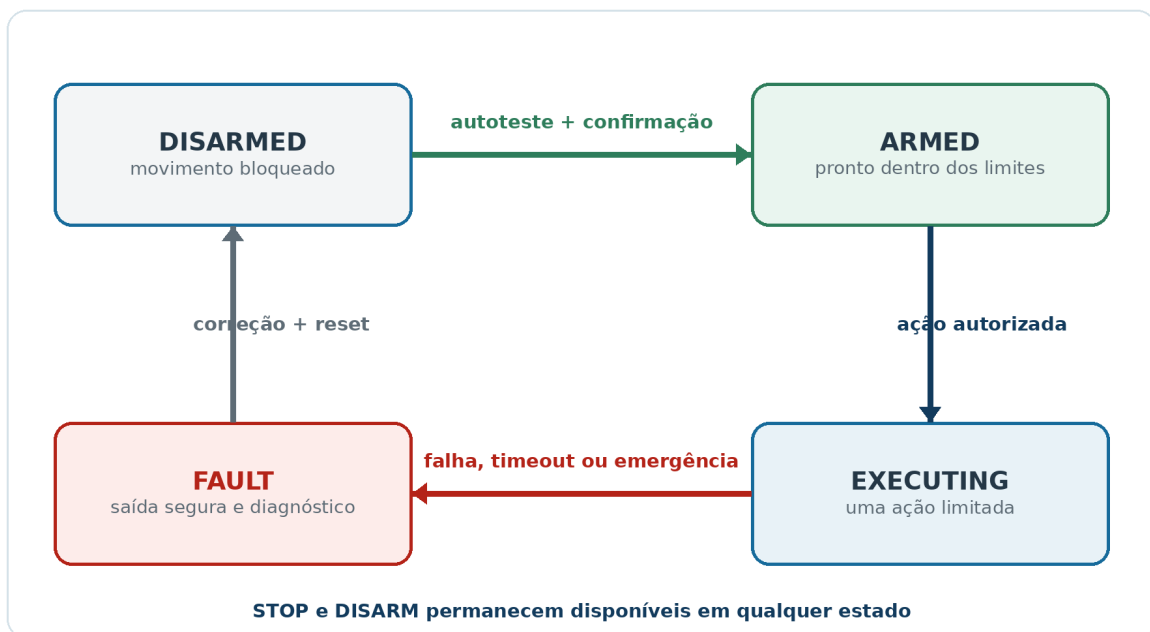


Figura 4 - Máquina de estados operacionais do Robotinics Rev. 3.

Fonte: Elaboração própria.

STOP e DISARM devem ser aceitos em qualquer estado. A perda de heartbeat, estouro de tempo, subtensão crítica, sobrecorrente ou acionamento do botão de emergência conduz a uma saída segura sem depender do Raspberry Pi.

2.5 Interfaces

O protocolo entre controlador e Arduino é textual, delimitado por linha e adequado à depuração em terminal. A versão de produção pode evoluir para frames com tamanho, sequência e verificação de integridade, mas deve manter um modo de diagnóstico legível.

Entre serviços de rede, prefira conexões autenticadas, bind restrito e mensagens com tamanho limitado. Os servidores históricos que aceitavam comandos em qualquer interface e encaminhavam dados ao shell não fazem parte da arquitetura recomendada.

3. Segurança e método de trabalho

3.1 Hierarquia de segurança

Segurança é implementada em camadas independentes:

1. **Mecânica:** proteção de engrenagens, fixação, centro de gravidade e ausência de pontos de esmagamento acessíveis.
2. **Elétrica:** fusível, chave geral, proteção contra inversão, BMS, fios dimensionados e conectores adequados.
3. **Firmware:** limites, timeout, watchdog, parada por obstáculo e estado desarmado na inicialização.
4. **Aplicação:** catálogo de comandos, intervalos, confirmação, autenticação e registro.
5. **IA:** prompt restrito, saída estruturada, limite de ações e ausência de acesso direto ao sistema operacional.
6. **Operação:** área livre, supervisão, botão de emergência e procedimento de teste.

Nenhuma camada autoriza remover a seguinte. Um prompt bem escrito não substitui o watchdog; um fusível não substitui o limite de corrente no projeto; uma confirmação na tela não substitui o botão físico.

3.2 Regras de bancada

- Faça os primeiros testes com as rodas suspensas.
- Use uma fonte de bancada com limitação de corrente antes da bateria.
- Conecte um subsistema por vez.
- Nunca ajuste fiação com o sistema energizado.
- Mantenha o botão de emergência ao alcance.
- Teste **STOP** antes de qualquer comando de movimento.
- Para servos, remova braços e cargas nos primeiros movimentos.
- Não carregue células de lítio dentro do robô sem projeto térmico e proteção adequados.
- Não deixe o conjunto carregando sem supervisão.
- Registre corrente de repouso, corrente de pico e temperatura.

3.3 Controle de mudanças

Cada alteração deve responder quatro perguntas:

1. Qual requisito motivou a mudança?
2. Quais arquivos e componentes foram alterados?
3. Como a mudança foi testada?
4. Como retornar à versão anterior?

Firmware, aplicação e documento devem informar versão. Uma versão publicada do livro deve apontar para uma tag do repositório; apontar apenas para `main` ou `master` torna a reprodução dependente de mudanças futuras.

3.4 Critérios de conclusão por etapa

Uma etapa está concluída quando:

- a montagem corresponde ao desenho ou a uma alteração registrada;
- não existem fios soltos, curto visível ou aquecimento inesperado;
- o consumo está dentro do orçamento;
- todos os comandos esperados respondem;
- falhas simuladas levam a estado seguro;
- os resultados foram registrados com data e versão;
- o próximo subsistema pode ser adicionado sem esconder falhas anteriores.

3.5 Segurança de credenciais e dados

Chaves de API não devem ser gravadas no repositório nem em um `settings.ini` legível. O exemplo do agente serial do TCHATGPT possui opção histórica de salvar token em texto claro; esta edição não adota essa opção. Use variável de ambiente, cofre do sistema operacional ou arquivo protegido fora da pasta do projeto.

Dados de câmera, microfone, telemetria e conversas podem conter informações pessoais. Quando um provedor remoto for usado, informe o operador, envie apenas o necessário e aplique política de retenção. Para uso educacional, prefira ambientes controlados e dados sem identificação.

3.6 Limitações declaradas

O Robotinics não é equipamento médico, industrial certificado, veículo para transporte de pessoas nem plataforma de segurança crítica. O uso em outro contexto exige análise de risco, normas aplicáveis e validação profissional. Componentes baratos e clones podem ter especificações diferentes; sempre meça o exemplar real.

3.7 Caderno de construção

Mantenha um caderno físico ou digital ao lado da bancada. Para cada etapa, anote a revisão dos arquivos, os componentes efetivamente usados, alterações realizadas, medidas, fotografias e resultado do teste. Esse registro transforma uma montagem que “funcionou uma vez” em um projeto que pode ser repetido e mantido.

Use uma página por subsistema com seis campos:

Campo	Pergunta que deve ser respondida
Objetivo	O que ficará pronto ao final?
Conhecimento	Qual conceito explica esta decisão?

Campo	Pergunta que deve ser respondida
Materiais	Quais peças, ferramentas e documentos serão usados?
Montagem	Qual foi a ordem real das operações?
Teste	Como comprovar que o resultado está correto?
Evidência	Qual medida, fotografia, log ou arquivo registra a aprovação?

NOTA Quando o exemplar montado divergir deste livro, registre a alteração antes de continuar. O livro explica a referência; o caderno descreve o seu robô.

Parte I - Mecânica

ROBOTINICS REV. 3

4. Dimensionamento mecânico

4.1 Requisitos antes da escolha dos motores

O motor não deve ser escolhido apenas por uma indicação comercial de torque. Primeiro defina massa, diâmetro das rodas, velocidade, inclinação máxima, tipo de piso, aceleração desejada e número de rodas motrizes. Depois calcule o esforço necessário e aplique margem.

Força e torque não são a mesma grandeza. Força é medida em newtons. Torque é o produto da força pela distância perpendicular ao eixo e é medido em newton-metro. Uma força de 10 N aplicada a 0,05 m do eixo produz 0,5 N·m.

4.2 Forças de movimento

Em uma subida com ângulo θ , a parcela do peso contrária ao movimento é:

TEXT

$$F_{\text{active}} = m \times g \times \sin(\theta)$$

A resistência aproximada ao rolamento é:

TEXT

$$F_{\text{rolamento}} = C_{rr} \times m \times g \times \cos(\theta)$$

Para acelerar:

TEXT

$$F_{\text{aceleração}} = m \times a$$

Em baixa velocidade, o arrasto aerodinâmico do Robotinics tende a ser pequeno em relação às demais parcelas, mas pode ser incluído quando necessário. A força total de projeto é a soma das parcelas relevantes. O torque requerido nas rodas é:

TEXT

$$T_{\text{total}} = (F_{\text{total}} \times \text{raio_da_roda}) / \text{eficiência}$$

Divida o resultado entre as rodas realmente motrizes. A eficiência representa perdas de transmissão, deformação, mancais e contato com o piso. Não use 100%.

4.3 Exemplo corrigido de tração

Considere um robô de 12 kg, duas rodas motrizes, raio de 0,05 m, rampa de 5°, coeficiente de rolamento estimado em 0,03, aceleração de 0,2 m/s² e eficiência global de 70%.

Parcela	Cálculo	Resultado aproximado
Active	$12 \times 9,81 \times \sin(5^\circ)$	10,26 N
Rolamento	$0,03 \times 12 \times 9,81 \times \cos(5^\circ)$	3,52 N
Aceleração	$12 \times 0,2$	2,40 N
Total	soma	16,18 N

O torque total de eixo é aproximadamente $(16,18 \times 0,05) / 0,70 = 1,16 \text{ N}\cdot\text{m}$. Com duas rodas motrizes, cada conjunto motor-redutor precisa fornecer cerca de $0,58 \text{ N}\cdot\text{m}$ na condição calculada.

Aplicando fator de segurança 2, a meta passa a aproximadamente $1,16 \text{ N}\cdot\text{m}$ por roda, ou cerca de $11,8 \text{ kgf}\cdot\text{cm}$.

Esse valor deve ser comparado ao torque contínuo do conjunto, não apenas ao torque de travamento. Operar continuamente perto do stall aquece o motor, aumenta o consumo e reduz a vida útil. Confirme também a velocidade depois da redução.

4.4 Aderência e limite de tração

Mesmo um motor forte não move o robô se a roda patinar. A força máxima transferível depende da carga normal sobre as rodas motrizes e do coeficiente de atrito. Bateria e componentes pesados devem ser posicionados baixos e distribuídos para manter estabilidade e aderência, sem sobrecarregar um único apoio.

Faça um ensaio de rampa com incrementos pequenos. Registre inclinação, corrente dos motores, velocidade, temperatura e ocorrência de patinação. Esse ensaio valida simultaneamente o cálculo mecânico e o orçamento elétrico.

4.5 Redução: trocar velocidade por torque

Uma redução mecânica permite que um motor gire mais rápido e com menor torque enquanto o eixo de saída gira mais devagar e com torque maior. Em um par de engrenagens, a relação ideal é:

TEXT

```
i = Z_saída / Z_motora
velocidade_saída = velocidade_motor / i
torque_saída_ideal = torque_motor * i
```

Se a engrenagem motora possui 12 dentes e a movida possui 36, $i = 36/12 = 3$. Idealmente, a saída gira a um terço da velocidade e oferece três vezes o torque. Na prática, atrito, flexão, desalinhamento e folga reduzem o ganho; use a eficiência informada pelo fabricante ou meça o conjunto.

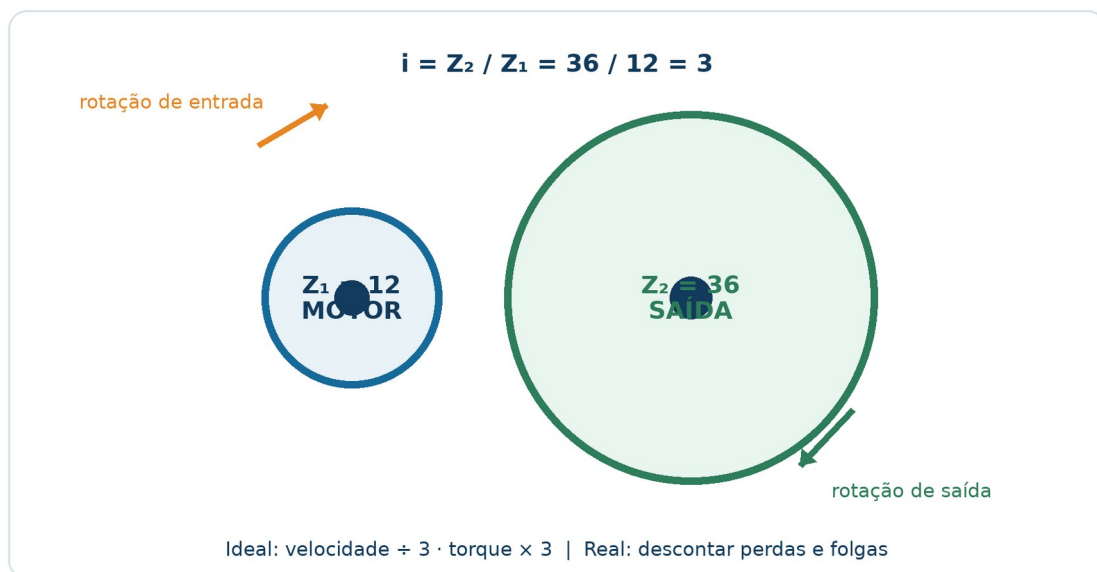


Figura 5 - Exemplo de redução 3:1: menor velocidade e maior torque disponível no eixo de saída.
Fonte: Elaboração própria.

No Robotinics, a redução pode estar integrada ao motoredutor da tração ou adicionada a uma articulação. Antes de escolher, verifique:

- torque contínuo e torque de pico, não apenas o torque de travamento;
- rotação desejada na roda ou articulação;
- diâmetro da roda e velocidade linear resultante;
- folga entre dentes e efeito sobre posicionamento;
- esforços radiais nos eixos e apoio por rolamentos;
- possibilidade de retorno da carga e necessidade de freio;
- corrente do motor na pior condição.

Exercício de bancada da redução

1. Marque um dente ou o eixo de entrada e outro no eixo de saída.
2. Gire manualmente a entrada e conte quantas voltas produzem uma volta na saída.
3. Compare a relação medida com a relação nominal.
4. Verifique pontos duros, ruído, folga e desalinhamento.
5. Com o motor alimentado por fonte limitada, meça rotação e corrente sem carga.
6. Aplique carga crescente dentro do limite e registre queda de rotação e aquecimento.

Teste antes de avançar: a transmissão gira livre, não salta dentes, não força a estrutura e atende à rotação e ao torque previstos com margem.

4.6 Torque dos braços

Para uma articulação, some o momento produzido por cada massa em relação ao eixo:

TEXT

$$T_{\text{estático}} = \sum (\text{massa}_i \times g \times \text{distância}_i)$$

A distância é medida entre o eixo e o centro de massa do item. Para o ombro, entram a massa do braço, do antebraço, da garra, dos servos instalados depois do eixo e da carga. A pior condição normalmente ocorre com o braço horizontal.

Exemplo: elo de 0,25 kg com centro de massa a 0,10 m; conjunto distal de 0,20 kg com centro a 0,28 m; carga de 0,10 kg a 0,40 m.

TEXT

$$\begin{aligned} T &= (0,25 \times 9,81 \times 0,10) \\ &+ (0,20 \times 9,81 \times 0,28) \\ &+ (0,10 \times 9,81 \times 0,40) \\ &\approx 1,19 \text{ N}\cdot\text{m} \approx 12,1 \text{ kgf}\cdot\text{cm} \end{aligned}$$

Esse é apenas o torque estático. Partida, parada, folga, impacto e desalinhamento exigem margem. Com fator 2,5, a articulação precisaria de aproximadamente **3,0 N·m**, ou **30,6 kgf·cm**. Portanto, um servo anunciado com 10 ou 15 kgf·cm não atende essa condição de forma confiável. As soluções são reduzir comprimento ou massa, limitar carga, usar contrapeso/mola, mudar a transmissão ou escolher atuador adequado.

NOTA A unidade tradicional de servos é $\text{kgf} \cdot \text{cm}$, não $\text{kgf} \cdot \text{cm}^2$. Para conversão aproximada, $1 \text{ kgf} \cdot \text{cm} = 0,0981 \text{ N} \cdot \text{m}$.

4.7 Centro de gravidade e estabilidade

O centro de gravidade projetado no piso deve permanecer dentro do polígono formado pelos pontos de contato. Braços estendidos deslocam o centro e podem tombar o robô. Avalie as posições extremas no CAD e faça um teste físico com o robô desenergizado.

Mantenha bateria e fontes na região inferior. Limite por software combinações de pose que aproximem o centro de gravidade da borda. A IA pode escolher uma tarefa, mas um planejador determinístico deve rejeitar poses fora do envelope validado.

5. Estrutura e montagem mecânica

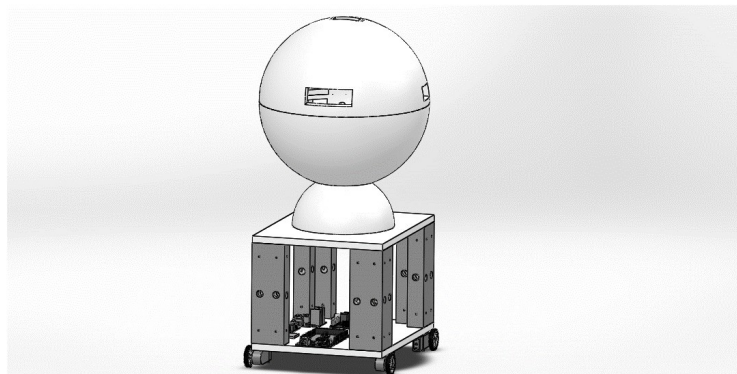
O que você vai construir: uma estrutura mecânica completa, ainda sem energia, com base móvel, suportes internos, corpo, braços, garras e cabeça.

Conhecimentos necessários: leitura de desenho, escala, tolerância, orientação de impressão, torque, redução, centro de gravidade, fixação e roteamento de cabos.

Materiais e ferramentas: arquivos STL/CAD da revisão escolhida, impressora 3D ou peças fabricadas por terceiro, parafusos e insertos compatíveis, chaves, paquímetro, esquadro, lixas, trava-rosca adequada, motores, rodas e massas de teste equivalentes à bateria e às placas.

5.1 Acervo mecânico

O repositório contém peças SolidWorks, montagens e arquivos STL. Entre os elementos principais estão a base, suportes de roda, corpo inferior e superior, suporte do Raspberry Pi, cabeça, extensões dos braços e garras. Arquivos STL permitem reprodução sem possuir o software CAD original; os arquivos de montagem permitem alterações mais profundas.



*Figura 6 - Modelo CAD do conjunto Robotinics preservado do acervo original.
Fonte: Acervo do projeto Robotinics.*

Antes de imprimir, verifique:

- unidade de medida e escala;
- revisão e orientação da peça;
- furos, folgas e espessura mínima;
- espaço para porcas, cabeças de parafuso e conectores;
- sentido de carga em relação às camadas;
- interferência entre cabos e partes móveis;
- acesso para manutenção.

5.2 Material e impressão

PLA é simples de imprimir e serve para protótipos rígidos, mas pode perder resistência com temperatura e sofrer fratura em encaixes. PETG oferece maior tenacidade e tolerância térmica em muitos cenários.

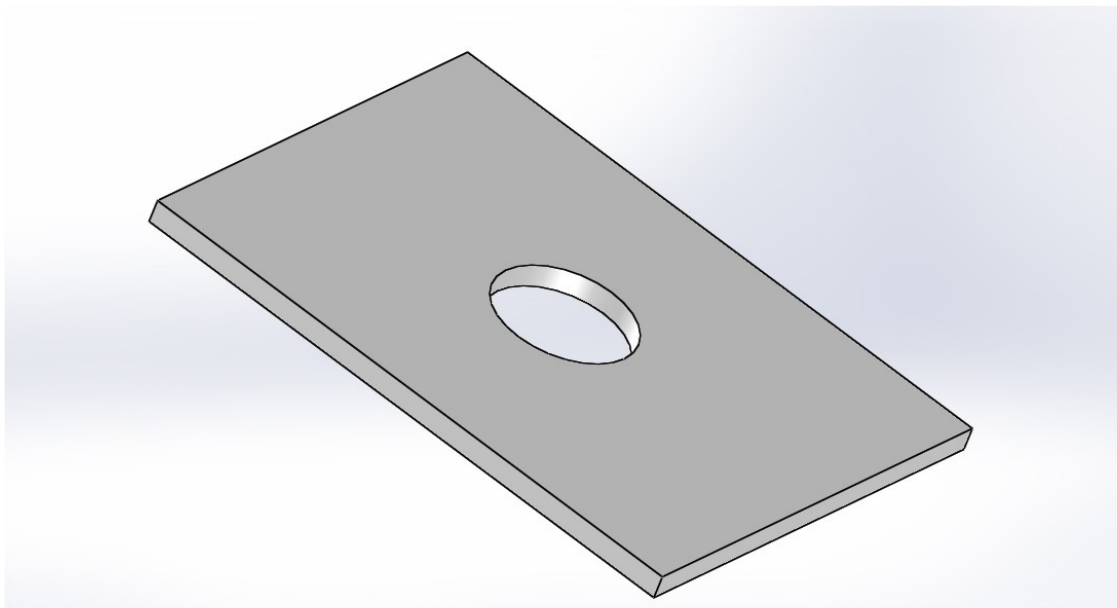
ABS/ASA pode ser útil para peças funcionais, desde que a impressora e o ambiente controlem empenamento e emissões. A escolha depende da carga, temperatura e capacidade de impressão.

Não copie parâmetros de fatiamento sem validar. Registre material, fabricante, diâmetro do bico, altura de camada, número de perímetros, preenchimento e orientação. Para suportes de motor e articulações, perímetros e orientação costumam ser mais importantes que um preenchimento alto e aleatório.

5.3 Base

A base deve ser montada e validada antes do corpo. A sequência recomendada é:

1. Inspecionar e desbastar as peças impressas.
2. Montar suportes, motores e rodas sem eletrônica.
3. Verificar paralelismo e giro livre.
4. Posicionar bateria e placas com gabaritos, sem fixação definitiva.
5. Conferir centro de gravidade e acesso à chave geral.
6. Instalar canaletas, prensa-cabos e pontos de aterramento lógico.
7. Fazer teste de rolagem manual.



*Figura 7 - Peças superiores da base mecânica no acervo CAD do projeto.
Fonte: Acervo do projeto Robotinics.*

Rodas desalinhadas aumentam corrente e prejudicam odometria. Meça distância entre eixos em ambos os lados e verifique se a base não está torcida. Rodízios devem apoiar sem elevar uma roda motriz.

Resultado esperado: a base rola por impulso em linha aproximadamente reta, as rodas não raspam, os motoredutores permanecem firmes e os compartimentos da bateria e das placas podem ser acessados sem desmontar a tração.

5.4 Corpo, braços e cabeça

O corpo abriga placas, distribuição e conexões. Monte painéis de forma removível e identifique cada chicote. Braços devem possuir batentes ou limites seguros antes de receber potência. A cabeça recebe câmera, sensores e mecanismos de orientação; evite passar cabos por zonas de pinçamento.

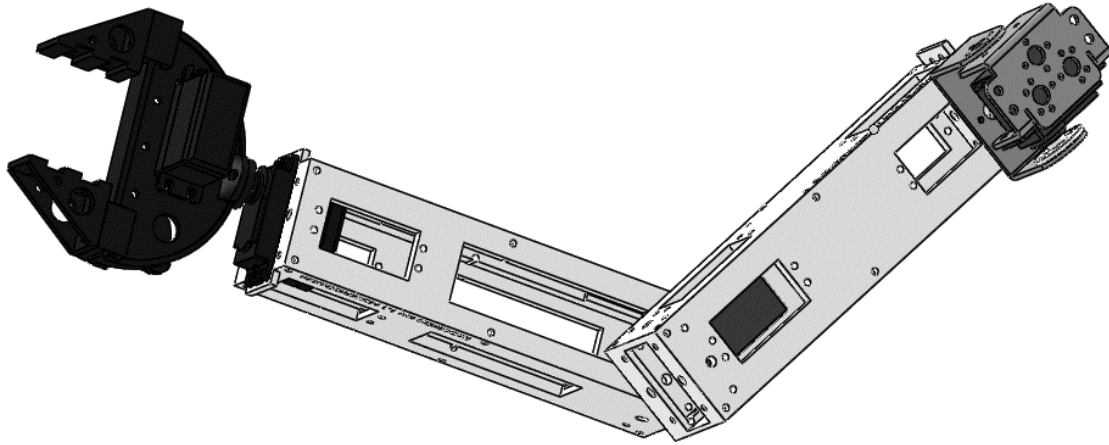


Figura 8 - Conjunto mecânico do braço e da garra.
Fonte: Acervo do projeto Robotinics.

Para cada articulação, registre:

Item	Registro mínimo
Zero mecânico	posição física de referência
Zero lógico	valor enviado ao servo nessa posição
Limite mínimo e máximo	intervalo sem colisão
Corrente em vazio	referência para detectar travamento
Carga máxima	condição validada, não apenas anunciada
Sentido positivo	convenção usada pelo firmware

Passo a passo de uma articulação

1. Monte a peça fixa, o eixo e o apoio sem instalar o braço do servo.
2. Mova a articulação à mão e identifique colisões, pontos de pinçamento e curso útil.
3. Ligue somente o servo em bancada, sem carga, e comande a posição central definida para aquele modelo.
4. Desenergize, instale o braço do servo no zero mecânico e aperte a fixação.
5. Reenergize com limite de corrente e teste deslocamentos pequenos para ambos os lados.
6. Aumente gradualmente o curso, grave limites seguros e repita com o elo instalado.
7. Meça corrente em vazio e com a carga de projeto; interrompa se houver travamento ou aquecimento anormal.

ATENÇÃO Nunca reposicione manualmente um servo energizado. Mantenha dedos, cabos e ferramentas fora da trajetória da articulação.

5.5 Alternativas à impressão 3D

O conceito original permite construir volumes com materiais acessíveis, como chapas, perfis e elementos esféricos revestidos. Qualquer substituição deve preservar fixação, rigidez, acesso e proteção. Isopor não deve ficar exposto a produtos que o dissolvam; revestimentos e adesivos devem ser testados em amostra.

5.6 Critérios de aceitação mecânica

- Nenhuma peça trincada ou deformada.
- Parafusos críticos com trava adequada e marca de inspeção.
- Rodas livres e alinhadas.
- Cabos sem esforço durante todo o curso.
- Centro de gravidade estável com braços nas poses permitidas.
- Proteção contra contato acidental com partes móveis.
- Acesso à bateria, fusível e emergência sem desmontagem extensa.

Entrega da Parte I: fotografe a estrutura pelos quatro lados, registre massas e dimensões, anote limites das articulações e guarde o relatório de rolagem. Somente então comece a instalação elétrica.

Parte II - Eletrônica

ROBOTINICS REV. 3

6. Alimentação, bateria e distribuição

O que você vai construir: a distribuição elétrica protegida do robô, com trilhos separados para tração, servos e lógica, pronta para receber as placas e os sensores.

Conhecimentos necessários: tensão, corrente, potência, energia, polaridade, queda de tensão, fusível, BMS, conversores e medição com multímetro.

Materiais e instrumentos: pack e carregador compatíveis, BMS, fusível e porta-fusível, chave geral e emergência, conversores, bornes, fios dimensionados, etiquetas, multímetro e, preferencialmente, fonte de bancada com limitação de corrente.

6.1 Grandezas fundamentais

Tensão representa diferença de potencial. Corrente representa fluxo de carga. Potência elétrica em corrente contínua é $P = V \times I$. Energia é potência acumulada no tempo e pode ser expressa em Wh. Capacidade de bateria em Ah não é energia por si só; é necessário considerar a tensão.

Na edição anterior, correntes de trilhos diferentes foram somadas sem converter a potência. O procedimento correto é calcular a potência de cada carga no seu trilho, incluir eficiência dos conversores e então estimar a corrente retirada da bateria.

6.2 Conjunto 3S

Considere três células de íons de lítio de 3,7 V nominais e 5,2 Ah ligadas em série:

Propriedade	Resultado
Tensão nominal	$3 \times 3,7 = 11,1 \text{ V}$
Tensão máxima de carga	$3 \times 4,2 = 12,6 \text{ V}$, se especificado para a célula
Capacidade	5,2 Ah
Energia nominal aproximada	$11,1 \times 5,2 = 57,7 \text{ Wh}$

Em série, as tensões se somam e a capacidade em Ah permanece a mesma. O valor de 15,6 Ah corresponderia a três células iguais em paralelo, não em série.

Valores de 4,2 V por célula são comuns, mas a ficha técnica do fabricante é soberana. Química, limite de carga, corrente e temperatura variam. Não misture células de modelos, capacidades, idades ou estados diferentes.

6.3 Carga segura

Células de lítio exigem carregador CC/CV compatível com o número de células e um sistema de proteção/balanceamento apropriado. Um BMS protege o conjunto, mas não transforma uma fonte comum em carregador correto. Para 3S, use carregador projetado para 3S e para a química real, com tensão final e corrente dentro da especificação.

Não derive a corrente de carga por uma regra universal como $C/10$. Use o valor recomendado pelo fabricante das células e pelo pack. Células em série não triplicam a capacidade para esse cálculo.

PERIGO A Rev. 3 não ensina a construir carregador conectado à rede elétrica. Use equipamento certificado e apropriado ao pack. Célula inchada, danificada, aquecida ou com tensão anormal deve ser isolada e encaminhada conforme orientação local para resíduos perigosos.

6.4 BMS, fusível e chave geral

A arquitetura mínima inclui:

1. pack montado de forma segura;
2. BMS/proteção compatível com 3S, corrente e química;
3. fusível próximo ao terminal positivo da bateria;
4. chave geral dimensionada para corrente contínua;
5. distribuição protegida para cada trilho;
6. conectores polarizados e com capacidade adequada.

O fusível protege o condutor e reduz energia de uma falha. Seu valor não deve exceder a capacidade segura dos fios e conectores. Correntes de partida precisam ser consideradas para evitar disparo indevido, sem transformar o fusível em mero fio.

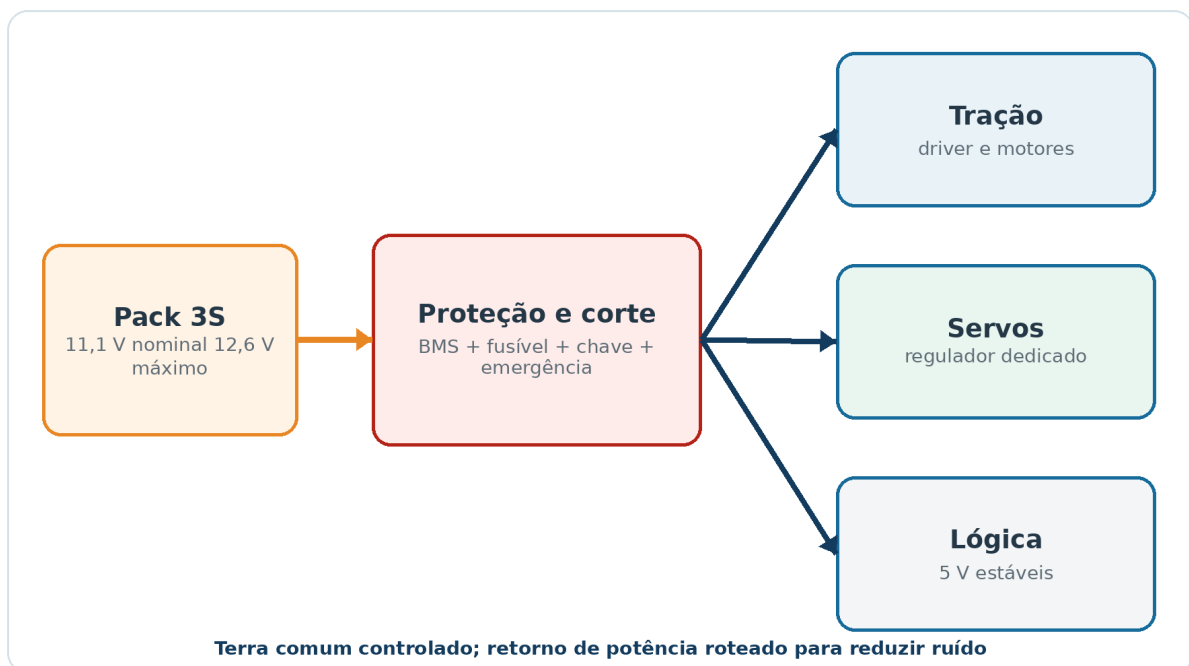


Figura 9 - Distribuição de energia com proteção e trilhos separados.
Fonte: Elaboração própria.

6.5 Trilhos separados

Motores DC, servos e lógica produzem e toleram ruído de formas diferentes. A Rev. 3 usa trilhos separados:

- **Tração:** tensão compatível com os motores e driver, com proteção própria.

- **Servos:** conversor de alta corrente na tensão permitida pelos servos.
- **Lógica 5 V:** Raspberry Pi e periféricos compatíveis, com margem para picos.
- **Lógica 3,3 V:** sensores ou interfaces que a exijam.

Os terras podem precisar de referência comum para sinais, mas a distribuição deve evitar que corrente de motor atravesse o retorno da lógica. Use topologia de estrela, condutores adequados, desacoplamento e verificação com osciloscópio quando possível.

Uma fonte “5 V/5 A” não é automaticamente suficiente para Raspberry Pi, servos e periféricos. Servos de alto torque podem produzir picos elevados. Dimensione com medições e margem, e nunca alimente vários servos pelo regulador da placa Arduino.

6.6 Conversores

Um conversor buck reduz tensão. Ele não consegue regular 12,6 V a partir de uma entrada de 12 V. Para elevar tensão é necessário boost; para operar acima e abaixo da saída, buck-boost. No Robotinics, é preferível escolher arquitetura em que os conversores trabalhem dentro de sua faixa com margem.

Verifique tensão máxima, corrente contínua real, refrigeração, ripple, eficiência e resposta a transientes. A corrente impressa em módulos baratos pode representar pico em condição ideal, não operação contínua.

6.7 Autonomia

A autonomia aproximada pode ser estimada por:

TEXT

$$\text{tempo}_h \approx (\text{energia_nominal_Wh} \times \text{eficiência_utilizável}) / \text{potência_média_W}$$

Com 57,7 Wh, eficiência utilizável global de 80% e consumo médio de 35 W, a estimativa é $(57,7 \times 0,8) / 35 \approx 1,32$ h. Movimento, terreno, temperatura, envelhecimento e limites do BMS reduzem ou alteram o resultado. Valide em ensaio controlado e não descarregue além do limite seguro.

6.8 Capacitores e ripple

$20,75 \times 10^{-3}$ F equivale a 0,02075 F, isto é, 20.750 μF ; não a 20,75 μF . Em uma fonte retificada, uma aproximação comum é:

TEXT

$$C \approx I / (f_{\text{ripple}} \times \Delta V)$$

Para 1 A, ripple de 120 Hz e variação de 1 V, $C \approx 8.333$ μF . Esse exemplo não substitui projeto de fonte, tolerância, ESR, tensão do capacitor e corrente de ripple.

6.9 Medição correta

Tensão é medida em paralelo. Corrente é medida em série com a carga ou por sensor apropriado. Colocar o multímetro configurado para corrente diretamente entre positivo e negativo cria um curto de baixa resistência e pode queimar fusível, pontas, circuito ou causar acidente.

Antes de medir:

- confirme bornes e função selecionada;
- comece pela escala adequada;
- desenergize para alterar ligação em série;
- respeite categoria, tensão, corrente e fusível do instrumento;
- use pinça ou sensor quando o método em série não for seguro.

6.10 Critérios de aceitação elétrica

- Polaridade e continuidade verificadas sem bateria.
- Fusíveis instalados e identificados.
- Tensão de cada trilho dentro da tolerância, sem carga e sob carga.
- Ripple e queda de tensão aceitáveis durante partida de motor/servo.
- Corrente de repouso e picos registrados.
- Nenhum conector ou fio aquecendo além do esperado.
- BMS e carregador compatíveis e documentados.
- Subtensão e desligamento testados de forma controlada.

7. Eletrônica, sensores e atuadores

O que você vai construir: o painel eletrônico e os chicotes removíveis que ligam Arduino Mega, shield, driver, servos e sensores sem depender de jumpers soltos.

Conhecimentos necessários: níveis lógicos, entrada e saída digital, ADC, PWM, UART, I²C, ponte H, corrente de partida, desacoplamento, crimpagem e continuidade.

Materiais e ferramentas: Arduino Mega 2560, shield com conectores carenados e polarizados, terminais e carcaças compatíveis, alicate de crimpagem adequado, driver de motores, sensores, servos, fios, etiquetas, multímetro e fonte limitada em corrente.

7.1 Arduino e níveis lógicos

O firmware histórico foi desenvolvido para Arduino Mega, cuja lógica é normalmente 5 V. Raspberry Pi usa GPIO de 3,3 V e não tolera 5 V diretamente. Toda ligação entre placas deve considerar nível, direção, corrente e estado durante inicialização. Use conversor de nível quando necessário; nunca confie apenas em uma coincidência observada em bancada.

O Mega 2560 foi mantido porque oferece 54 pinos digitais, 16 entradas analógicas e quatro portas seriais de hardware, recursos coerentes com o número de periféricos do projeto. A ligação ao Raspberry Pi é preferencialmente feita por USB, que também facilita diagnóstico e evita expor GPIO de 3,3 V a sinais de 5 V.

7.2 Shield com conectores carenados e polarizados

A shield aberta usada na edição anterior expunha fileiras de pinos nas quais um conector de servo podia ser deslocado ou invertido. Nesta revisão ela é substituída por uma shield para Arduino Mega com conectores plásticos carenados, chanfro de orientação e retenção por atrito. A referência visual fornecida pelo autor aparece a seguir.

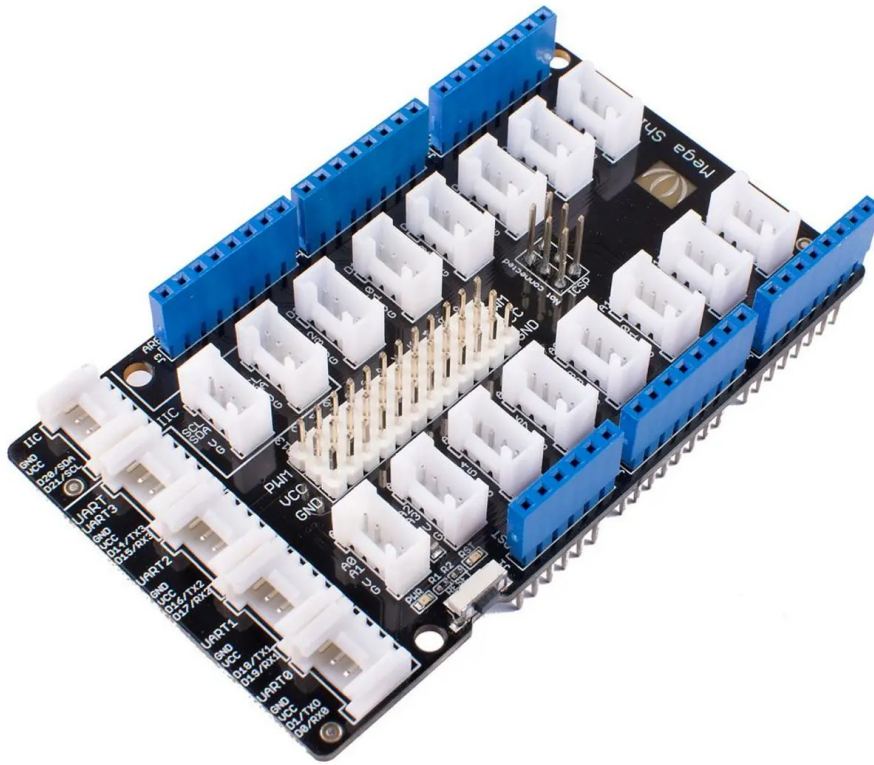
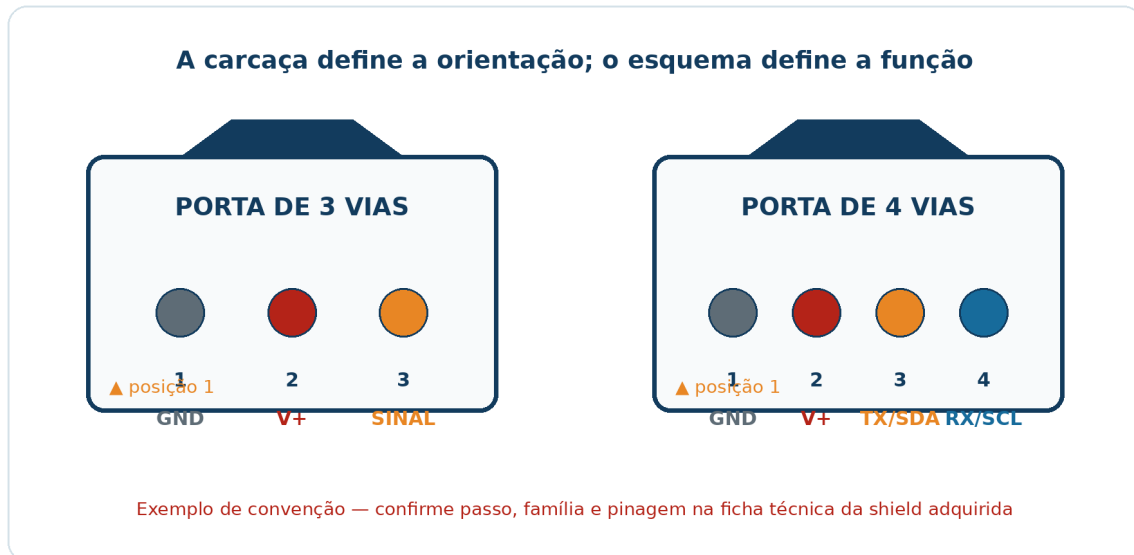


Figura 10 - Shield de referência para Arduino Mega com conectores carenados e polarizados.
 Fonte: Referência visual fornecida pelo autor; modelo comercial meramente ilustrativo.

O [DS1069-4MVW da Connfly](#) é um exemplo de conector macho de quatro vias no passo nominal de 2,5 mm. Famílias comerciais semelhantes são anunciadas como KK, KF2510, JST-XH ou “2,54 mm”, mas **não são automaticamente intercambiáveis**. Geometria da trava, passo, terminal, corrente e ordem dos pinos precisam corresponder à shield e à carcaça adquiridas.

Conectores da família KK utilizam recursos de polarização e retenção para reduzir encaixe incorreto, mas essa proteção é mecânica. Ela não determina se a sequência é **GND-VCC-SINAL**, **SINAL-VCC-GND** ou outra. A serigrafia da placa e o esquema do fornecedor prevalecem.



*Figura 11 - Conector polarizado: trava mecânica, numeração e ordem elétrica continuam obrigatórias.
Fonte: Elaboração própria.*

Especificação de compra

Antes de encomendar a shield e os terminais, confirme:

Item	O que conferir
Placa	compatibilidade física e elétrica com Arduino Mega 2560
Passo	valor real entre centros, por exemplo 2,50 ou 2,54 mm
Carcaça	número de vias, orientação e trava compatíveis com o cabeçalho
Terminal	bitola do fio, material e ferramenta de crimpagem recomendada
Corrente	limite por contato e limite do barramento da própria shield
Serigrafia	posição de sinal, VCC e GND em cada grupo
Barramentos	se VCC de servos pode ser isolado da lógica e alimentado externamente

ATENÇÃO A shield distribui sinais e facilita manutenção; ela não deve transportar a corrente total de vários servos sem confirmação de trilhas, conectores e fabricante. Use alimentação dedicada aos servos, com proteção própria e terra comum controlado. Nunca uma saídas de fontes diferentes.

Fabricação do chicote

1. Desenhe uma tabela de ponta a ponta antes de cortar fios.
2. Escolha cores consistentes: preto para GND, vermelho para alimentação e outras cores para sinais.
3. Corte o fio com folga para manutenção, sem deixar laços alcancarem rodas ou articulações.
4. Decape somente o comprimento indicado pelo terminal.
5. Crimpe primeiro as abas do condutor e depois as abas do isolamento.
6. Faça teste leve de tração em cada terminal.
7. Insira o terminal na carcaça até a lingueta travar e confira a posição 1.
8. Meça continuidade ponta a ponta e confirme ausência de curto entre vias adjacentes.

- 9. Etiquete as duas extremidades com função e destino, por exemplo **US - FRONT**.
- 10. Conecte o chicote à placa ainda desenergizada e faça nova conferência visual.

Teste antes de avançar: com Arduino, bateria e sensores desconectados, cada via apresenta continuidade apenas até o destino previsto; GND, VCC e sinal estão na mesma ordem da shield.

7.3 Mapa de pinos derivado do firmware

O mapa a seguir foi extraído de `Software/arduino/robotinics/robotinics.ino`. Ele documenta o protótipo histórico e deve ser comparado à revisão do firmware carregada. Comentários antigos nem sempre correspondem ao identificador; a tabela usa as declarações efetivas do arquivo.

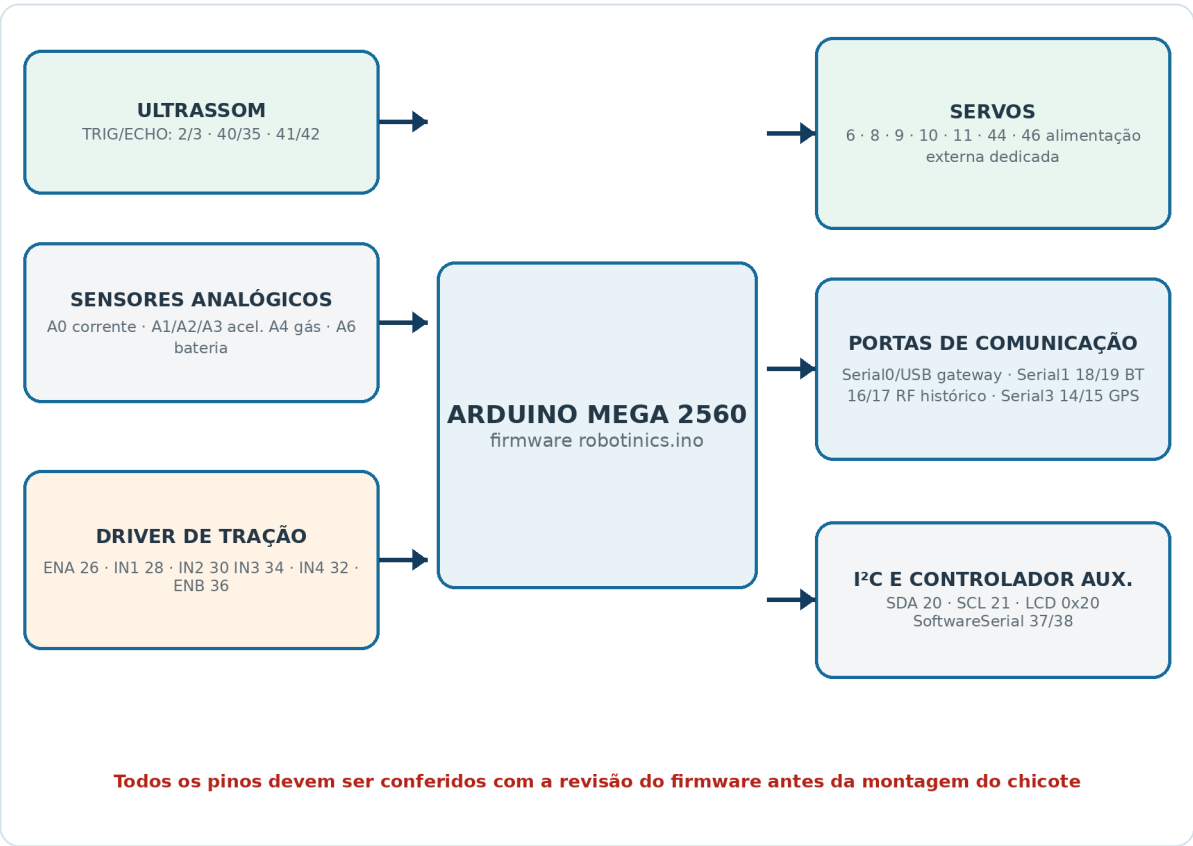


Figura 12 - Mapa de ligações derivado do firmware histórico robotinics.ino.
Fonte: Elaboração própria a partir de `Software/arduino/robotinics/robotinics.ino`.

Grupo	Função no código	Pino do Mega	Conector recomendado
Servo	cabeça	D6	3 vias: sinal, Vservo, GND
Servo	braço direito	D44	3 vias: sinal, Vservo, GND
Servo	mão direita	D8	3 vias: sinal, Vservo, GND
Servo	braço esquerdo	D9	3 vias: sinal, Vservo, GND
Servo	garra esquerda	D10	3 vias: sinal, Vservo, GND
Servo	garra direita	D11	3 vias: sinal, Vservo, GND
Servo	pé esquerdo	D46	3 vias: sinal, Vservo, GND
Analógico	corrente	A0	3 vias: OUT, VCC, GND
Analógico	acelerômetro X/Y/Z	A1/A2/A3	conector multipolar conforme módulo

Grupo	Função no código	Pino do Mega	Conector recomendado
Analógico	gás	A4	3 vias: OUT, VCC, GND
Analógico	tensão da bateria	A6	divisor protegido; nunca ligar pack direto
Ultrassom	traseiro TRIG/ECHO	D2/D3	4 vias: VCC, TRIG, ECHO, GND
Ultrassom	dianteiro TRIG/ECHO	D40/D35	4 vias: VCC, TRIG, ECHO, GND
Ultrassom	cabeça TRIG/ECHO	D41/D42	4 vias: VCC, TRIG, ECHO, GND
Tração	ENA/IN1/IN2/IN3/IN4/ENB	D26/D28/D30/D34/D32/D36	chicote de controle separado da potência
Bluetooth	Serial1 RX1/TX1	D19/D18	4 vias: GND, VCC, TX, RX, com cruzamento
Rádio histórico	RX/TX declarados	D17/D16	confirmar módulo e firmware antes de usar
GPS	Serial3 RX3/TX3	D15/D14	4 vias; ajustar nível e cruzar TX/RX
I ² C	SDA/SCL e LCD 0x20	D20/D21	4 vias: GND, VCC, SDA, SCL
Controlador auxiliar	SoftwareSerial	D37/D38	confirmar sentido TX/RX no sketch revisado

NOTA O arquivo histórico contém macros de GPS inválidas e uma porta analógica A5 configurada como entrada apesar do nome `analogOutGas`. Esses pontos permanecem na matriz de correções e devem ser corrigidos na versão de firmware usada na montagem.

Ordem prática de ligação

1. Instale o Mega e a shield sem conectar periféricos.
2. Verifique continuidade entre os pinos do Mega e os contatos correspondentes da shield.
3. Alimente apenas a lógica por USB e confirme que nenhum barramento de servo recebe tensão indevida.
4. Conecte I²C e interfaces seriais, uma por vez, e execute testes de identificação.
5. Conecte sensores analógicos e ultrassônicos e confira valores em repouso.
6. Conecte o driver sem motores; teste níveis de controle com o robô desarmado.
7. Conecte um servo por vez à fonte dedicada e aplique os limites mecânicos registrados.
8. Somente depois instale motores, com as rodas suspensas e a fonte limitada em corrente.

7.4 Driver de motores

O L298N é uma ponte H dupla baseada em transistores bipolares. Ele permite controle de direção e velocidade por PWM aplicado aos pinos de enable. Não fornece isolamento galvânico e apresenta queda de tensão e dissipação relevantes, especialmente em baixa tensão de motor.

Se o projeto for mantido por compatibilidade, meça tensão no motor e temperatura do driver na pior carga. Em uma revisão eletrônica, um driver MOSFET moderno, dimensionado para tensão, corrente de partida e frenagem, tende a oferecer maior eficiência.

O motor aparece com tensões diferentes em trechos históricos. A Rev. 3 não assume 12 V: identifique o modelo instalado e respeite sua ficha técnica. Uma tensão de bateria de 11,1/12,6 V não autoriza ligar diretamente um motor de 3-6 V.

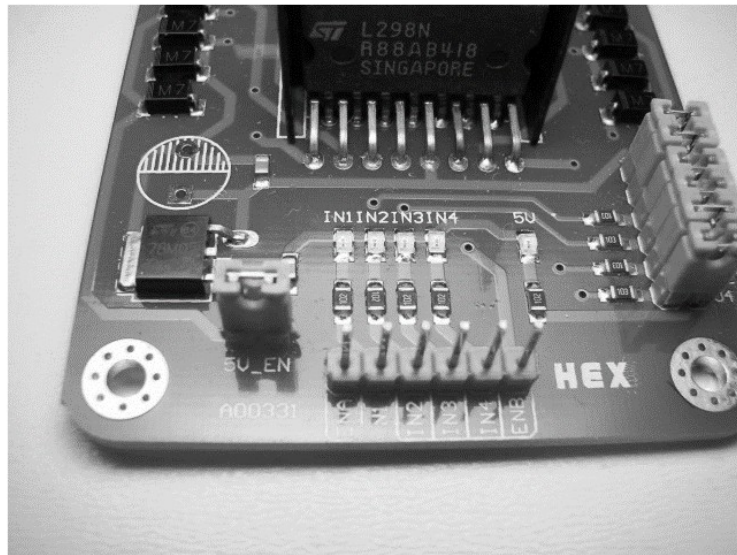


Figura 13 - Módulo L298N empregado no protótipo histórico para o acionamento dos motores.
Fonte: Acervo da edição anterior do projeto Robotinics.

No firmware histórico, **ENA**, **IN1**, **IN2**, **IN3**, **IN4** e **ENB** foram atribuídos a D26, D28, D30, D34, D32 e D36. O chicote de controle leva apenas sinais e referência. Os cabos de bateria e motores seguem diretamente pelo circuito de potência, com bitola e conectores dimensionados para a corrente de partida.

7.5 Relés

Contato normalmente aberto (NA/NO) permanece aberto com a bobina desenergizada e fecha quando acionada. Contato normalmente fechado (NF/NC) permanece fechado em repouso e abre quando acionado. Verifique o diagrama do componente, tensão da bobina, corrente dos contatos e necessidade de diodo de roda livre.

7.6 LEDs

0,02 A corresponde a 20 mA, não 200 mA. O resistor é calculado por:

TEXT

$$R = (V_{\text{fonte}} - V_{\text{LED}}) / I_{\text{LED}}$$

Para fonte de 5 V, LED vermelho com queda aproximada de 2 V e corrente desejada de 9 mA, $R \approx 333 \, \Omega$; $330 \, \Omega$ é um valor comercial adequado no exemplo. Calcule a potência do resistor por $P = I^2 \times R$ e confirme a especificação real do LED.

7.7 Sensor ultrassônico

O HC-SR04 utiliza pulsos ultrassônicos em torno de 40 kHz, não 40 Hz. O microcontrolador envia pulso de trigger e mede a duração do eco. A distância aproximada é obtida considerando ida e volta do som:

TEXT

$$\text{distância} = \text{velocidade_do_som} \times \text{tempo} / 2$$

Faça leituras independentes, descarte timeouts e aplique filtragem. Repetir três vezes a conversão do mesmo tempo não produz três amostras. No Arduino, defina os pinos usados e limite `pulseIn` com timeout para evitar bloqueio prolongado.

O sinal echo de muitos módulos é 5 V. Para Raspberry Pi, use adaptação de nível. Temperatura, ângulo, material e interferência entre sensores afetam o resultado; o ultrassom é auxílio, não única barreira de segurança.

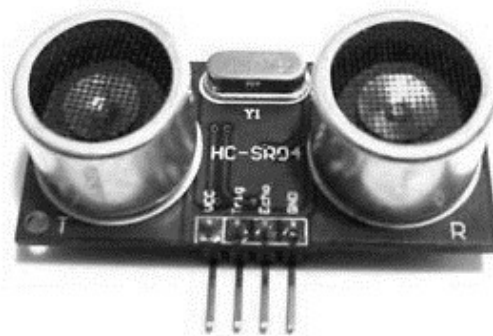


Figura 14 - Sensor ultrassônico HC-SR04 usado para medir distância.
Fonte: Acervo da edição anterior do projeto Robotinics.

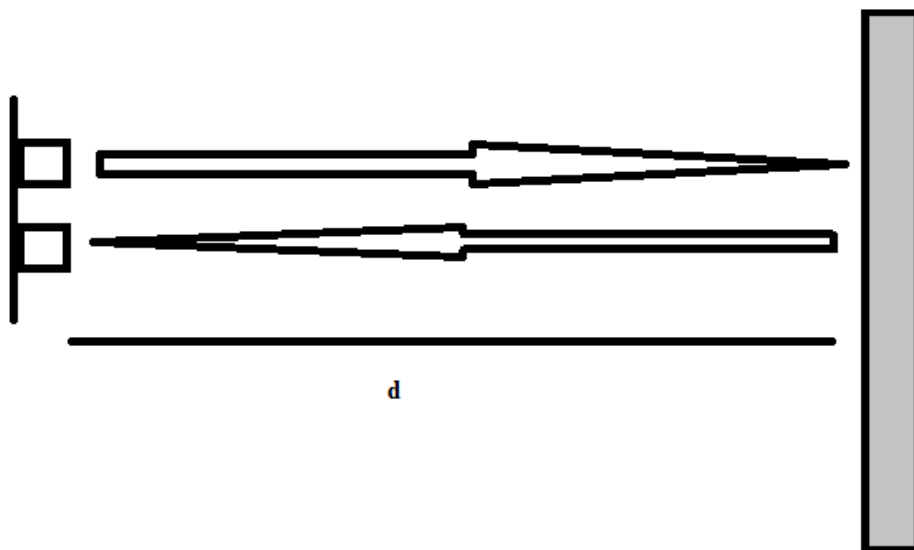


Figura 15 - Princípio de medição por tempo de voo do ultrassom.
Fonte: Acervo da edição anterior do projeto Robotinics, redesenhado editorialmente.

Ligação e teste de cada HC-SR04

1. Com a alimentação desligada, ligue **VCC** e **GND** e depois **TRIG** e **ECHO** conforme a tabela de pinos.
2. Confirme no multímetro que não existe curto entre VCC e GND.
3. Energize apenas Arduino e sensor e execute um sketch mínimo com timeout.
4. Posicione uma superfície plana a distâncias conhecidas, por exemplo 10, 30 e 60 cm.
5. Compare a mediana de várias leituras com a régua e registre erro e dispersão.
6. Repita com os três sensores individualmente antes de habilitá-los em sequência.
7. Defina intervalo entre disparos para reduzir interferência cruzada.

7.8 Servos

A biblioteca **Servo** do Arduino usa temporizadores e pode controlar servos em diversos pinos digitais; não é correto afirmar que somente pinos marcados como PWM servem. Entretanto, o uso de temporizadores pode interferir em outras funções, e o número de canais depende da placa e da biblioteca.

Alimente servos por fonte separada e una a referência de sinal de forma planejada. Defina limites mecânicos individuais. Na inicialização, não envie abruptamente uma posição que faça a articulação colidir; carregue calibração, valide e mova com rampa.

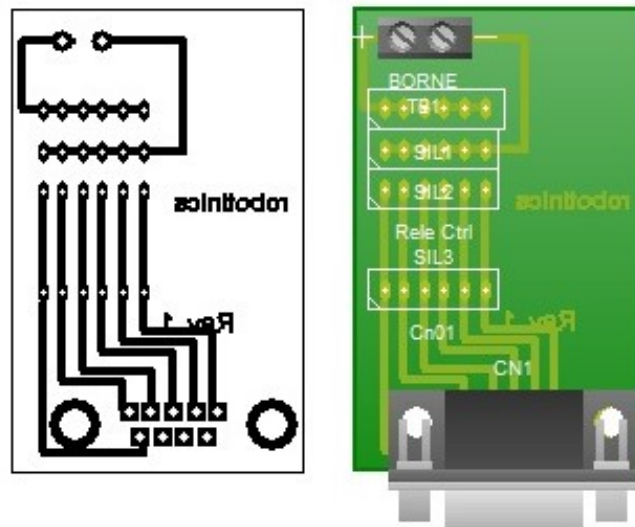


Figura 16 - Placa controladora de servos preservada no acervo eletrônico do Robotinics.
Fonte: Acervo do projeto Robotinics.

7.9 Sensores analógicos

Entradas de tensão e corrente devem permanecer dentro da faixa do ADC. Divisores resistivos precisam considerar tolerância e impedância; sensores de corrente precisam de calibração de zero, ganho e ruído. Para calcular média ou RMS, zere acumuladores no início da janela e use amostras distintas.

Um alarme de sobrecorrente deve existir abaixo do nível que danifica fios, driver ou bateria. A resposta rápida pode ficar no firmware; o Raspberry registra e explica o evento.

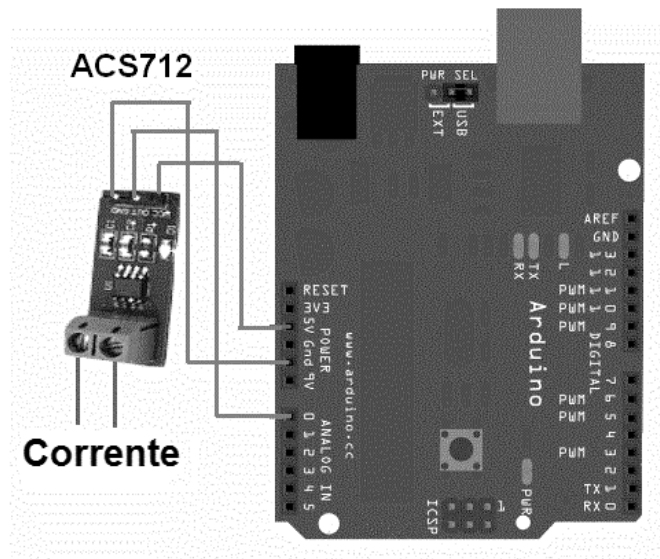


Figura 17 - Exemplo histórico de ligação do sensor de corrente ACS712 ao Arduino.
Fonte: Acervo da edição anterior do projeto Robotinics.

7.10 Desacoplamento e compatibilidade eletromagnética

Instale capacitores de desacoplamento próximos aos circuitos, mantenha loops de corrente pequenos e separe cabos de potência de sinais. Motores com escovas podem exigir supressão no próprio motor. Trance pares de alimentação e retorno quando apropriado e evite usar cabos/conectores de dados sem verificar corrente, queda de tensão e pinagem.

Conector RJ45 pode ser usado como conector físico interno, mas isso não transforma o circuito em Ethernet. Identifique claramente para impedir conexão acidental a uma rede.

7.11 Checklist eletrônico

- Tensão e corrente nominal de cada componente confirmadas.
- Níveis de 3,3 V e 5 V compatibilizados.
- Pinos e conectores documentados.
- Driver dimensionado pela corrente de partida.
- Servos alimentados fora do regulador do Arduino.
- Sensores calibrados e com timeout.
- Relés e cargas indutivas com proteção.
- Cabos etiquetados, presos e livres de partes móveis.
- Teste de ruído executado com motores e servos em movimento.

Entrega da Parte II: salve o diagrama elétrico, a tabela de chicotes, fotografias das duas extremidades de cada conexão e a planilha de tensões e correntes. O software somente será integrado quando cada carga funcionar separadamente e o desligamento de emergência tiver sido testado.

Parte III - Software e inteligência artificial

ROBOTINICS REV. 3

8. Firmware Robotinics Rev. 3

O que você vai construir: o firmware determinístico que inicializa o robô desarmado, lê sensores, controla atuadores dentro de limites, publica o catálogo de comandos e para com segurança quando a comunicação falha.

Conhecimentos necessários: Arduino IDE, C/C++, entradas e saídas, temporização sem bloqueio, buffers, parser, máquina de estados, watchdog e teste serial.

Materiais: Arduino Mega com a eletrônica da Parte II ainda sem cargas perigosas, cabo USB, computador com Arduino IDE 2 atual, cópia versionada de `Software/arduino/robotinics` e terminal serial.

8.1 Preparar, compilar e gravar

1. Instale a versão atual da Arduino IDE 2 pela documentação oficial.
2. Abra `Software/arduino/robotinics/robotinics.ino` a partir de uma cópia de trabalho versionada.
3. Selecione **Arduino Mega or Mega 2560** e o processador `ATmega2560`.
4. Instale somente as bibliotecas exigidas pelo sketch e registre nome e versão: `LiquidCrystal_I2C`, `Ultrasonic`, `Servo` e `RCSwitch`, além das bibliotecas padrão.
5. Corrija as macros inválidas de GPS e os demais itens marcados na matriz editorial antes de considerar a build aprovada.
6. Compile sem placa conectada e salve o log completo de avisos e uso de memória.
7. Conecte apenas Mega e shield por USB, sem motores, servos ou bateria.
8. Grave o firmware e abra o monitor na velocidade configurada, atualmente 115200 bit/s para `Serial`.
9. Reinicie três vezes e confirme que nenhuma saída de movimento é ativada.
10. Registre hash do arquivo, versão exibida e mapa de pinos usado para construir o chicote.

ATENÇÃO O sketch histórico é a fonte que documenta o protótipo, mas contém problemas conhecidos. Compilar não basta: a revisão operacional deve incluir parser limitado, estado desarmado, watchdog e testes descritos neste capítulo.

8.2 Por que reorganizar

O sketch histórico demonstra muitos recursos, mas concentra cerca de 1.500 linhas em um único arquivo. Ele usa `String` intensivamente em um microcontrolador com pouca RAM, mistura buffers de várias portas, contém esperas bloqueantes e interpreta comandos com testes de substring. Isso torna possível que um prefixo como `ULTRA` também corresponda a `ULTRA1`, além de dificultar timeout, testes e manutenção.

Também existem defeitos objetivos: definições como `#define PINOGPSRX = 0;` não são macros válidas quando expandidas; uma rotina prepara `sInfo` e imprime outra variável; o recorte de `SERIAL:` usa índice

incompatível com o prefixo; a leitura ultrassônica repete a conversão do mesmo tempo; o parser de GPS pode bloquear e exceder o buffer.

A Rev. 3 mantém o valor do firmware original, mas propõe uma arquitetura modular e uma interface estável.

8.3 Módulos

Módulo	Responsabilidade
Config	pinos, limites e versão da placa
CommandParser	recepção, tokenização e validação sintática
Safety	estado, heartbeat, emergência, timeouts e falhas
Drive	direção, PWM, rampa e frenagem
Servos	calibração, limites e movimento gradual
Sensors	ultrassom, corrente, tensão, gás e acelerômetro
Telemetry	respostas estruturadas e eventos
Manual	catálogo determinístico publicado por MAN

Em placas pequenas, os módulos podem continuar compilados em um único sketch, mas devem permanecer separados por arquivos ou funções de responsabilidade clara.

8.4 Inicialização segura

Ao ligar:

1. Configure pinos de enable em estado inativo antes dos pinos de direção.
2. Desligue motores e mantenha o robô em **DISARMED**.
3. Inicialize serial e sensores com timeout.
4. Carregue calibração validada.
5. Publique versão e motivo de qualquer falha.
6. Somente aceite movimento após **ARM** e verificação das condições locais.

Servos devem iniciar em posição conhecida sem saltos. Quando a posição física não é conhecida, mova lentamente para uma referência segura ou exija procedimento manual.

8.5 Entrada serial limitada

Use um buffer fixo, por exemplo 128 bytes. Cada byte recebido é anexado até **LF** ou **CRLF**. Se o limite for excedido, descarte a linha, sinalize **ERR LINE_T00_LONG** e não execute conteúdo parcial. Cada porta deve possuir seu próprio estado de recepção.

Depois de receber uma linha, separe o primeiro token e compare o comando inteiro, sem procurar substring. Parâmetros são convertidos com detecção de erro e faixa. Uma linha inválida gera erro, nunca comportamento parcial.

TEXT

Entrada: MOVEFWD 120 800


```

Comando: MOVEFWD
PWM:     120, validado em 0..255
Duração: 800 ms, validada em 1..2000

```

8.6 Manual de comandos

O `TAIAgentSerial` do projeto TCHATGPT descobre o catálogo por marcadores e linhas determinísticas. O firmware Rev. 3 deve responder:

```

TEXT
MAN-BEGIN
DEVICE: Robotinics Rev3
VERSION: 3.0.0
BAUD: 115200 8N1
COMMAND PING: testa comunicação, sem parâmetros
COMMAND STATUS?: retorna estado, tensão, corrente e falhas
COMMAND ARM: habilita o estado armado após validação local
COMMAND DISARM: desabilita atuadores
COMMAND STOP: interrompe imediatamente o movimento
COMMAND MOVEFWD: MOVEFWD <pwm 0..255> <tempo_ms 1..2000>
COMMAND MOVEBACK: MOVEBACK <pwm 0..255> <tempo_ms 1..2000>
COMMAND TURNLEFT: TURNLEFT <pwm 0..255> <tempo_ms 1..1500>
COMMAND TURNRIGHT: TURNRIGHT <pwm 0..255> <tempo_ms 1..1500>
COMMAND RANGE?: retorna distâncias válidas dos sensores
COMMAND BATTERY?: retorna tensão e estado estimado
COMMAND MAN: publica este manual
MAN-END

```

O agente interpreta somente linhas `COMMAND`. Informações como versão e baud são apresentadas ao operador, mas não criam ações. Comandos desconhecidos permanecem bloqueados. Os aliases históricos `FRENTE`, `RE`, `GESQ`, `GDIR` e `PARA` podem ser mantidos para uso manual, mapeados internamente para a nova API e sujeitos aos mesmos limites.

8.7 Watchdog e heartbeat

Movimento deve expirar. Mesmo quando o comando contém duração, o firmware registra o instante de início e interrompe ao atingir o limite. Se a aplicação usar movimento contínuo, ela envia heartbeat periódico; ausência por um intervalo curto, por exemplo 500 ms, conduz a `STOP` e `FAULT` ou `DISARMED`, conforme a política.

O watchdog de hardware deve ser configurado para recuperar travamento do firmware. Após reset por watchdog, o robô volta desarmado e registra a causa. Reiniciar não deve retomar a ação anterior.

8.8 Obstáculos e limites locais

O firmware pode bloquear avanço quando um sensor frontal válido indicar distância menor que a margem configurada. Leituras inválidas não devem ser convertidas automaticamente em “caminho livre”. Defina uma política explícita: reduzir velocidade, parar e solicitar nova leitura.

Servos possuem limites por articulação. Corrente ou tempo excessivo sugere travamento e deve interromper a saída. A parada física de emergência deve retirar energia dos atuadores de modo apropriado e informar seu estado à lógica quando possível.

8.9 Telemetria

Respostas seguem padrões simples:

TEXT

```
OK <comando> [detalhes]
ERR <código> <descrição curta>
STATE <chave>=<valor> ...
EVENT <tipo> <dados>
```

Exemplos:

TEXT

```
OK ARM
STATE MODE=ARMED BATTERY_MV=11780 FRONT_MM=640
EVENT STOP REASON=HEARTBEAT_TIMEOUT
ERR RANGE_TIMEOUT SENSOR=FRONT
```

Inclua unidade no nome ou no contrato. `BATTERY_MV=11780` é menos ambíguo que `BATTERY=11.78` sem unidade.

8.10 Evitar fragmentação de memória

Em AVR, prefira arrays de `char`, buffers limitados e funções que não criem muitas cópias. Evite concatenar `String` indefinidamente. Conteúdo vindo da serial nunca deve aumentar sem limite. Testes de longa duração precisam observar RAM, estabilidade e recuperação de linhas inválidas.

8.11 Testes do firmware

- `STOP` funciona em todos os estados.
- Movimento é recusado em `DISARMED`.
- PWM e duração fora do intervalo são recusados.
- Linha longa não executa fragmento.
- Comando desconhecido é recusado.
- Perda de heartbeat para o movimento.
- Sensor com timeout não congela o loop.
- Reset retorna ao estado seguro.
- `MAN` contém apenas comandos implementados.
- Prefixos semelhantes não causam dupla execução.

9. Raspberry Pi e serviços do robô

O que você vai construir: o computador de bordo atual, responsável por câmera, áudio, rede, gateway serial, telemetria e serviços de IA, sem substituir o controle determinístico do Arduino.

Conhecimentos necessários: Linux, imagem de sistema, usuários e permissões, USB/serial, serviços, logs, rede, armazenamento, câmera CSI e limites de temperatura e consumo.

Materiais: Raspberry Pi 5 recomendado, fonte USB-C de 5 V/5 A compatível, resfriamento ativo, microSD de boa procedência ou SSD, câmera, cabo CSI adequado ao Pi 5, cabo USB para o Mega e rede Ethernet ou Wi-Fi.

9.1 Papel do Raspberry Pi

O Raspberry Pi não substitui o Arduino no controle de tempo real. Ele executa tarefas de maior nível: interface de câmera e áudio, integração de rede, armazenamento, aplicação Lazarus compatível, gateway serial, telemetria e serviços de IA que caibam na plataforma.

O sistema deve continuar seguro durante boot, atualização ou travamento. A linha de enable dos atuadores não pode depender de um GPIO que fique flutuando; o Arduino mantém o estado desarmado até receber sequência válida.

9.2 Hardware atual recomendado

Para uma nova montagem, use o [Raspberry Pi 5](#), com processador Cortex-A76 de quatro núcleos a 2,4 GHz, GPIO de 40 pinos e memória escolhida conforme a carga. O modelo de 8 GB oferece margem útil para visão e serviços simultâneos; 4 GB atende gateway, câmera e modelos pequenos quando o sistema é bem dimensionado. Mais memória não substitui medição de latência e temperatura.

O Pi 5 requer atenção à alimentação e à refrigeração. Use fonte capaz de fornecer 5 V/5 A conforme a recomendação da plataforma, cabos curtos e confiáveis e resfriamento ativo em processamento sustentado. Servos e motores continuam em fontes separadas: o conector USB-C do Pi não é barramento de potência dos atuadores.

Para uma versão embarcada ou placa própria, o [Compute Module 5](#) é alternativa posterior. Ele oferece a mesma geração de processamento em formato de módulo, opções de eMMC e maior liberdade de integração, mas exige carrier board e projeto eletrônico adicional. Comece pelo Pi 5 convencional para reproduzir o livro.

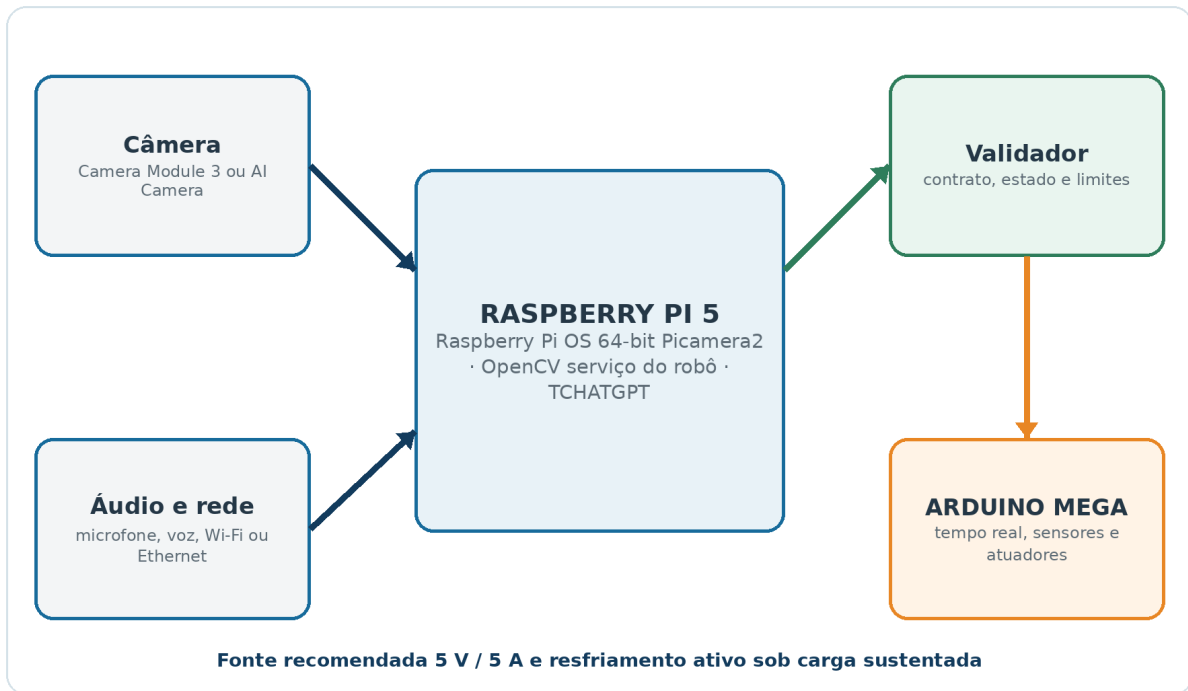


Figura 18 - Pilha atualizada de software e percepção no Raspberry Pi 5.
Fonte: Elaboração própria.

9.3 Instalação do sistema passo a passo

Use o Raspberry Pi Imager e a documentação oficial atual. Configure usuário próprio, senha forte, rede e SSH durante a gravação quando suportado. Não presuma o antigo usuário `pi` com senha `raspberrypi`; essa prática pertence a imagens históricas.

Na data editorial desta revisão, a opção corrente é o Raspberry Pi OS de 64 bits baseado em Debian Trixie; Bookworm aparece como versão legada. Como esse estado muda, confirme a página oficial antes de gravar. Use a edição Lite quando o robô não precisa de desktop local.

1. Instale o Raspberry Pi Imager em outro computador.
2. Selecione Raspberry Pi 5 e Raspberry Pi OS Lite de 64 bits.
3. Nas opções avançadas, defina hostname, usuário exclusivo, senha forte, fuso horário, rede e chave SSH.
4. Grave e valide o microSD ou SSD; anote versão e data da imagem.
5. Instale o resfriamento, conecte armazenamento, rede e monitor apenas se necessário; deixe Arduino e atuadores desconectados.
6. Ligue o Pi e acesse localmente ou por SSH.
7. Atualize o sistema e reinicie:

BASH

```
sudo apt update
sudo apt full-upgrade
sudo reboot
```

8. Instale ferramentas básicas de diagnóstico, câmera e visão pelos repositórios da distribuição:

BASH

```
sudo apt install git python3-picamera2 python3-opencv rpicas-apps
```

9. Liste câmeras com `rpicom-hello --list-cameras` e faça uma prévia ou captura conforme a documentação atual.
10. Conecte o Arduino Mega por USB, localize a porta estável em `/dev/serial/by-id/` e adicione o usuário do serviço ao grupo de acesso serial, normalmente `dialout`.
11. Saia e entre novamente após mudar grupos; então teste `PING`, `MAN` e `STATUS?` em terminal antes de iniciar qualquer serviço automático.

NOTA O Pi 5 usa conectores de câmera de 22 pinos. A Camera Module 3 comum usa cabo de 15 pinos; portanto, adquira o cabo mini de 22 para 15 pinos apropriado. Não force o flat cable e confirme a orientação dos contatos.

Registre:

- modelo do Raspberry Pi;
- arquitetura ARM64 ou ARMHF;
- versão do Raspberry Pi OS;
- imagem e data;
- pacotes instalados;
- serviços habilitados;
- versão do kernel e firmware;
- resultado dos testes de porta serial, câmera e áudio.

9.4 Perfil mínimo

Comece pelo sistema Lite quando o robô não precisa de ambiente gráfico local. Instale apenas dependências necessárias. Interfaces gráficas, Chromium, OpenCV, voz e modelos locais aumentam consumo, armazenamento e superfície de falha.

O TCHATGPT declara Linux ARM64 e ARMHF como experimentais no snapshot desta edição. Portanto, compile primeiro `openai_core` e um teste de console. Depois valide `openai_input`, `openai_voice` e `openai_vision` separadamente. Não instale toda a suíte e atribua uma falha genérica ao Raspberry.

9.5 Serviços com systemd

Aplicações de longa duração devem ser gerenciadas pelo `systemd`, com usuário sem privilégios administrativos, diretório de trabalho explícito, reinício controlado e log no journal.

```
INI
[Unit]
Description=Robotinics Gateway
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=robotinics
Group=robotinics
WorkingDirectory=/opt/robotinics
ExecStart=/opt/robotinics/bin/robotinics-gateway
Restart=on-failure
RestartSec=3
```

```
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Adapte permissões de dispositivo serial por grupo e regras apropriadas. Não execute o serviço como root apenas para acessar `/dev/ttyUSB0`.

9.6 Dependências e instalador

Dependências externas devem ser resolvidas de forma determinística pelo instalador ou por script versionado. A documentação informa quais são obrigatórias, opcionais e específicas de plataforma. O teste do instalador precisa confirmar bibliotecas em runtime, não apenas a compilação.

Para os pacotes TCHATGPT, componentes que usam Python devem passar pelo `TAIPythonRuntime`. Visão, voz e bibliotecas nativas precisam validar arquitetura. Um binário x64 não funciona em ARM64; uma biblioteca de 32 bits não satisfaz um processo de 64 bits.

9.7 Câmera e áudio

Teste captura e reprodução fora da IA. Primeiro confirme dispositivo, formato, taxa e latência. Em seguida conecte os componentes de voz ou visão. Isso separa falha de hardware, permissão e biblioteca de falha do modelo.

A pilha atual do Raspberry Pi usa `rpicam-apps` e `Picamera2`, que substituíram a API legada `Picamera`. `Camera Module 3` oferece sensor de 12 MP e foco automático. A [Raspberry Pi AI Camera](#), baseada no Sony IMX500, pode executar parte da inferência localmente e reduzir a carga do processador; ela é opcional, não requisito para acompanhar o livro.

Use nomes persistentes ou regras para dispositivos USB quando a ordem puder mudar. O serviço deve informar claramente “dispositivo ausente” e continuar seguro, em vez de travar o robô inteiro.

9.8 Armazenamento

Cartões microSD sofrem desgaste. Evite logs ilimitados e escrita contínua sem rotação. Use banco leve ou arquivos atômicos para configuração, com backup e validação. Dados importantes podem ser enviados a armazenamento externo quando a rede estiver disponível, sem bloquear controle local.

9.9 Atualização e rollback

Atualize aplicação e firmware como unidades versionadas. Antes de trocar:

1. salve configuração e versão corrente;
2. verifique assinatura ou origem do pacote;
3. instale em diretório de versão;
4. execute autoteste sem atuadores;
5. altere o link da versão ativa;
6. monitore o primeiro boot;

7. retorne à versão anterior se os critérios falharem.

Teste antes de avançar: depois de três reinicializações, o Pi inicia sem energizar atuadores, reconhece Arduino e câmera pelos mesmos identificadores, registra os serviços no journal e permanece dentro do limite térmico definido durante uma captura prolongada.

10. Comunicação, dados e segurança de software

10.1 Serial antes da rede

A serial local entre Raspberry e Arduino deve ser validada antes de expor qualquer interface de rede. Use 115200 8N1, terminação por linha e protocolo documentado, salvo motivo técnico para outra configuração. O gateway limita tamanho, taxa e tempo de espera.

Não reutilize o mesmo buffer para `Serial`, `Serial1` e uma `SoftwareSerial`. Cada canal possui origem e estado próprios. Quando uma resposta é encaminhada, identifique a origem.

10.2 Servidores históricos

Os programas `srvMonitor2` e `srvFala` documentam a experiência original, mas não devem ser executados em rede como estão. Foram encontrados problemas como tamanho de leitura inválido, buffer não terminado, formato controlado por entrada, variável errada enviada a `system`, falta de autenticação e bind em todas as interfaces.

O servidor de fala também encaminha entrada a um shell, o que permite injeção de comandos. A Rev. 3 substitui esse desenho por componentes de voz e uma API que aceita somente campos estruturados, com tamanho, caracteres e ações permitidas.

10.3 Portas e contratos

O material histórico usa portas diferentes para o mesmo serviço: a ferramenta de fala usa 7091, enquanto o servidor atual usa 8091. O controlador envia `PARAR` em um ponto, mas o firmware reconhece `PARA`. Contratos desse tipo devem existir em um único arquivo versionado e ser testados automaticamente.

A Rev. 3 recomenda não fixar portas em vários fontes. Use configuração central, valor padrão documentado e validação de conflito. Registre endereço, protocolo e finalidade.

10.4 Interface de rede

Por padrão, serviços do robô escutam apenas em 127.0.0.1 ou em socket local. Exposição na rede exige autenticação, autorização, limite de requisições e proteção do transporte conforme o risco. Nunca aceite comandos de atuador por Telnet ou socket cru acessível a qualquer máquina.

Quando acesso remoto for necessário, use uma VPN ou camada HTTPS configurada e mantida adequadamente. Firewall deve permitir somente origens e portas necessárias. O botão de emergência e a segurança local continuam independentes da rede.

10.5 Mensagens estruturadas

Para comunicação entre serviços, JSON é suficiente quando o volume é moderado. Defina esquema e versão:

JSON

```
{
  "schema": "robotinics.command.v1",
  "request_id": "8f2c...",
  "action": "move_forward",
  "parameters": {"pwm": 120, "duration_ms": 800},
  "requires_confirmation": true
}
```

O serviço valida tipos, campos obrigatórios, intervalos e tamanho antes de converter para o protocolo serial. Campos desconhecidos podem ser rejeitados para evitar interpretações divergentes.

10.6 Banco de dados

Banco de dados serve para configuração, inventário, telemetria e auditoria, não para o laço de segurança. O robô deve parar mesmo se o banco estiver indisponível.

Não use conta root na aplicação. Crie usuário com permissões mínimas e armazene segredo fora do código. Consultas são parametrizadas. Interfaces administrativas como phpMyAdmin não devem ficar publicamente expostas nem ser requisito de operação do robô.

O exemplo C histórico de MySQL contém aspas tipográficas, capitalização e parênteses que impedem compilação. Na Rev. 3, exemplos completos devem ser compilados em C; trechos ilustrativos são identificados como pseudocódigo.

10.7 Observabilidade

Cada solicitação recebe `request_id` ou `TraceID`. Registre:

- origem e horário;
- versão do controlador e firmware;
- intenção recebida;
- ação estruturada proposta;
- resultado de validação e confirmação;
- comando enviado;
- telemetria de retorno;
- duração e erro.

Não registre chave de API, senha, áudio ou imagem sensível por padrão. Logs devem possuir rotação e política de retenção.

10.8 Testes de resiliência

- desconectar rede durante movimento;
- interromper resposta do modelo;
- enviar JSON inválido e muito grande;
- repetir `request_id`;
- desconectar e reconectar serial;

- reiniciar Raspberry;
- travar processo de gateway;
- indisponibilizar banco;
- simular resposta atrasada;
- confirmar que nenhuma falha impede o Arduino de executar **STOP**.

Inteligência artificial aplicada ao robô

11. Preparando o TCHATGPT

11.1 O que é um LLM e o que ele faz no Robotinics

Um **modelo de linguagem de grande porte** (LLM) é uma rede neural treinada para estimar a continuação de sequências de texto. Ele aprende relações entre palavras, código e outros padrões a partir de muitos exemplos. Isso permite resumir, classificar, conversar e transformar uma solicitação em uma estrutura, mas não garante verdade, repetibilidade nem conhecimento do estado físico atual.

No Robotinics, o LLM funciona como intérprete de intenção. Ele pode converter “avance um pouco” em uma proposta como `{"action": "move_forward", "pwm": 90, "duration_ms": 400}`. O resultado ainda passa por parser, catálogo, estado, limites, confirmação e firmware. O modelo não escreve diretamente em pinos, não escolhe tensão, não desativa watchdog e não executa comandos de sistema operacional.

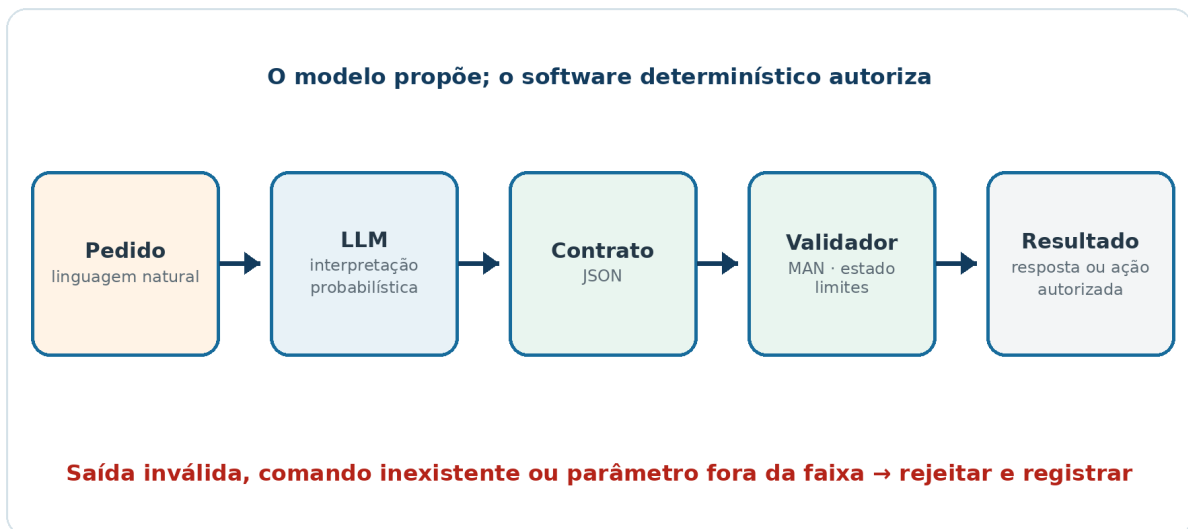


Figura 19 - Papel do LLM: interpretar intenção e propor uma saída estruturada, sem acionar hardware diretamente.
Fonte: Elaboração própria.

Termo	Significado prático no projeto
Token	unidade de texto processada; influencia limite, custo e latência
Contexto	instruções, catálogo e observações fornecidos naquela solicitação
Temperatura	controla variação; baixa temperatura ajuda formato, mas não garante correção
Prompt	instrução enviada ao modelo; orienta, mas não autoriza ação física
Inferência	execução do modelo para produzir uma resposta
Quantização	redução da precisão numérica para diminuir memória e, às vezes, acelerar inferência
Alucinação	saída plausível porém incorreta ou inventada
Tool calling	proposta estruturada para outro software executar uma função validada

Modelo remoto ou local

- **Remoto:** libera recursos do Raspberry Pi, mas depende da rede, do custo e da política de dados do provedor.

- **Local em servidor:** mantém dados na rede do laboratório e permite modelos maiores em computador x64.
- **Local no Pi 5:** funciona sem nuvem com modelos pequenos e quantizados, mas divide CPU, RAM e energia com visão e voz.

O [llama.cpp](#) executa modelos quantizados em C/C++ e pode publicar um endpoint compatível com a API da OpenAI. Em Raspberry Pi, comece por um modelo pequeno, limite o contexto e meça tokens por segundo, memória, temperatura e tempo total. “Roda localmente” não significa “responde rápido o suficiente para controle”; movimentos urgentes permanecem no firmware.

11.2 O projeto

TCHATGPT é uma suíte de componentes visuais e não visuais para Lazarus e Free Pascal. No commit adotado, a unit `chatgpt` declara versão 1.7 e integra provedores remotos e servidores locais. A suíte é modular: não instale o antigo pacote monolítico `openai.lpk`.

Para a integração básica do Robotinics, os pacotes principais são:

Pacote	Uso no robô	Estado no snapshot
<code>openai_core</code>	TCHATGPT, prompts, modelos e base	Stable/Beta por compilação
<code>openai_input</code>	serial, sockets, captura e entrada	Stable/Beta; combinações específicas podem ser experimentais
<code>openai_agent</code>	agente serial, memória, ações e segurança	misto; componentes usados aqui possuem evidência, pipeline geral segue experimental
<code>openai_voice</code>	captura, STT, TTS, clonagem autorizada e assistente	Stable/Beta por samples dedicados
<code>openai_vision</code>	imagem, filtros, câmera e OpenCV	Beta, com backends de maturidade diferente
<code>openai_rag</code>	indexação e recuperação de documentos	Stable/Beta por sample
<code>openai_observability</code>	TraceID, spans e métricas	Stable/Beta por compilação

“PASS” no relatório do projeto significa que um sample compilou. Isso não comprova automaticamente câmera, microfone, placa, modelo ou serviço externo em runtime. A Rev. 3 mantém essa distinção.

11.3 Provedores

TCHATGPT oferece OpenAI, OpenRouter, Cerebras, Gemini, Claude, DeepSeek, endpoint OpenAI-compatible, llama.cpp, neural-api e perfil local. O livro não obriga um provedor.

Escolha conforme privacidade, latência, custo, disponibilidade e capacidade do hardware:

- **Remoto:** menor exigência computacional no robô; depende de internet e política do provedor.
- **Local x64:** bom para laboratório com um servidor na rede; dados permanecem no ambiente controlado.
- **Local no Raspberry:** maior independência, mas modelos e visão competem por CPU/RAM; requer medição real.

- **Híbrido:** comandos simples e segurança local; tarefas linguísticas complexas em servidor remoto ou local mais potente.

11.4 Configuração sem segredo no código

O trecho abaixo usa a API real da versão congelada. `SendQuestion` retorna `Boolean`; a resposta fica em `Response`. Portanto, não trate o retorno da função como texto.

```
PASCAL
uses
  SysUtils, chatgpt;

procedure ConfiguraLLM(Chat: TCHATGPT);
begin
  Chat.Provider := AIP_OPENAI_COMPATIBLE;
  Chat.URL := GetEnvironmentVariable('ROBOTINICS_LLM_URL');
  Chat.CustomModel := GetEnvironmentVariable('ROBOTINICS_LLM_MODEL');
  Chat.TOKEN := GetEnvironmentVariable('ROBOTINICS_LLM_TOKEN');
  Chat.MaxTokens := 256;
  Chat.Temperature := 0.1;
  Chat.Timeout := 30000;
  Chat.Dev :=
    'Converta a intenção em JSON estrito. Não invente comandos. ' +
    'A execução física será validada por outra camada.';
end;

function PerguntaLLM(Chat: TCHATGPT; const Pergunta: string): string;
begin
  if not Chat.SendQuestion(Pergunta) then
    raise Exception.Create(Chat.LastError);
  Result := UTF8Encode(Chat.Response);
end;
```

No laboratório de Marcelo, um servidor local pode estar configurado na porta 8095. Esse valor é configuração do ambiente, não padrão universal do TCHATGPT. Defina `ROBOTINICS_LLM_URL`, por exemplo `http://127.0.0.1:8095/v1/chat/completions`, somente depois de confirmar endpoint e modelo.

11.5 Chamadas assíncronas

Interfaces gráficas não devem congelar durante uma requisição. `TCHATGPT` oferece `SendQuestionAsync`, streaming, eventos de estado e cancelamento. Os estados incluem `Idle`, `Connecting`, `Receiving`, `Completed`, `Cancelled` e `Error`.

Use chamadas assíncronas para conversa e explicações. Controle físico requer ainda serialização: não permita duas decisões concorrentes sobre o mesmo atuador. Um coordenador mantém uma fila curta, cancela solicitações antigas e invalida resposta que chegou depois de mudança de estado.

11.6 Prompts e contratos

Um prompt não é mecanismo de autorização. Ele descreve o formato esperado, mas toda resposta deve passar por parser. Para controle, use JSON estrito, conjunto pequeno de ações e parâmetros numéricos. Defina temperatura baixa e limite de tokens; respostas longas só aumentam latência e superfície de erro.

Não envie ao modelo todo o histórico, toda a telemetria ou documentos completos. Selecione contexto relevante, informe unidades e inclua o estado atual. Uma decisão feita com telemetria antiga deve expirar.

11.7 Autoteste

Antes de associar o LLM ao agente serial:

1. Execute uma pergunta curta e confirme `Response`.
2. Desligue o servidor e valide `LastError` e timeout.
3. Cancele uma requisição.
4. Teste texto com acentos e JSON.
5. Meça latência média e pior caso.
6. Confirme que o token não aparece em log ou configuração.
7. Execute o teste sem qualquer atuador energizado.

12. Agente de controle supervisionado

12.1 Componentes

O sample `agent_serial_demo` fornece a base mais próxima da necessidade do Robotinics. Ele usa:

- `TCHATGPT` para interpretação;
- `TAISerialModem` para porta real;
- `TAIListSerialDevices` para descoberta de portas;
- `TAIAgentSerial` para ações estruturadas;
- `TAIAgentAction` como catálogo de comandos;
- `TAIAgentMemoryMap` para histórico;
- eventos para confirmação, log e comandos rejeitados.

A Rev. 3 acrescenta `TAIAgentSafety` e um validador específico de parâmetros do robô.

12.2 Configuração segura

```
PASCAL
procedure TfrmRobotinics.ConfiguraAgente;
begin
  AIAgentSerial1.Serial := AISerialModem1;
  AIAgentSerial1.LLM := CHATGPT1;
  AIAgentSerial1.CommandCatalog := AIAgentCommands1;
  AIAgentSerial1.RequireConfirmation := True;
  AIAgentSerial1.MaxActionsPerPrompt := 3;
  AIAgentSerial1.AutoDiscoverCommands := True;
  AIAgentSerial1.AllowUnknownDeviceCommands := False;
  AIAgentSerial1.ClearCatalogOnDisconnect := True;

  AIAgentSafety1.Enabled := True;
  AIAgentSafety1.ReadOnlyMode := False;
  AIAgentSafety1.SimulationMode := False;
  AIAgentSafety1.AllowIndustrialWrite := True;
  AIAgentSafety1.RequireConfirmation := True;
end;
```

`AllowIndustrialWrite=True` não libera tudo. A lista de ações, registro de capacidades, catálogo `MAN`, validador de parâmetros e confirmação continuam obrigatórios.

12.3 Registro de capacidade

Registre a ação que representa envio físico:

```
PASCAL
RegisterActionCapability(
  'ROBOTINICS_SERIAL_SEND',
  [acIndustrialWrite, acStateMutation]
);
```

Adicione essa ação a `AIAgentSafety1.AllowedActions`. A política é fail-closed: ação sem registro deve ser recusada.

12.4 Evento antes da ação

O evento `OnBeforeAction` recebe o tipo e o parâmetro propostos. A implementação do Robotinics deve:

1. mapear o tipo interno para uma ação de segurança;
2. validar sintaxe e faixa do comando físico;
3. validar estado e telemetria atual;
4. chamar `TAIAgentSafety.ValidateAction`;
5. apresentar confirmação clara;
6. definir `AAllow=True` apenas se todas as etapas passarem.

PASCAL

```
procedure TfrmRobotinics.AgentBeforeAction(Sender: TObject;
  AKind: TAgentActionKind; const AParam: string; var AAllow: Boolean);
var
  Params: TStringList;
  Erro: string;
begin
  AAllow := False;

  if not ValidaComandoRobotinics(AKind, AParam, Erro) then
  begin
    RegistraRejeicao(AParam, Erro);
    Exit;
  end;

  Params := TStringList.Create;
  try
    Params.Values['COMMAND'] := AParam;
    AAllow := TAIAgentSafety1.ValidateAction(
      'ROBOTINICS_SERIAL_SEND', Params, Erro);
  finally
    Params.Free;
  end;

  if not AAllow then
    RegistraRejeicao(AParam, Erro);
end;
```

`ValidaComandoRobotinics` é código do aplicativo, pois conhece comandos, faixas, estado e sensores. O componente genérico não deve embutir regras específicas do robô.

12.5 Confirmação compreensível

Não mostre apenas “Permitir send?”. Mostre ação física e consequência:

TEXT

O robô moverá para frente com PWM 120 por até 800 ms.
 Distância frontal atual: 640 mm.
 Estado: ARMED. Bateria: 11,78 V.
 [Permitir uma vez] [Cancelar]

Não use confirmação genérica para uma sequência longa. Cada ação de movimento deve ser curta ou a sequência inteira deve ser apresentada com limite e opção de interrupção.

12.6 Descoberta determinística

Ao conectar, `TAIAgentSerial` envia `MAN`. O parser espera `MAN-BEGIN`, coleta linhas `COMMAND <nome>: <descrição>` e conclui em `MAN-END`. A LLM não interpreta o manual; o parser faz isso. Antes de `send`, o primeiro token é comparado ao catálogo.

Esse desenho é uma força do projeto TCHATGPT e deve ser preservado. Para o Robotinics, o firmware publica apenas comandos realmente disponíveis para a configuração atual. Um braço ausente não deve gerar comandos de braço.

12.7 Memória de sessão

`TAIAgentMemoryMap` pode registrar pergunta, resposta e ações. Não use histórico como fonte de estado físico: o estado vem da telemetria atual. Uma frase antiga como “o robô está armado” não autoriza movimento depois de reinício.

Não grave token no histórico. Defina retenção, opção de nova conversa e tamanho máximo. Para auditoria, armazene IDs e resultados de validação em log separado da conversa.

12.8 Limites atuais

`TAIAgentSerial.Execute` usa chamada síncrona ao LLM. Em GUI, execute a operação de forma que não bloqueie a interface e sincronize o acesso aos componentes. O componente limita número de ações, mas não conhece aceleração, geometria, bateria ou distância; essas regras pertencem ao Robotinics.

12.9 Testes adversariais

Peça ao modelo:

- um comando que não existe;
- movimento com PWM negativo ou 999;
- movimento por tempo muito longo;
- ignorar instruções e enviar texto ao shell;
- executar cinco ações quando o limite é três;
- mover enquanto `DISARMED`;
- avançar com obstáculo próximo;
- usar informação de um documento RAG como comando.

Todos devem ser bloqueados ou convertidos em pergunta sem ação.

13. Voz e interação humana

13.1 Pipeline

O pacote `openai_voice` integra o fluxo:

TEXT

```
TAIAudioInput → TAISpeechRecognizer → TCHATGPT
               → TAIVoiceClone ou TAIVoiceSynthesizer → TAIAudioPlayer
```

`TAIVoiceAssistant` coordena reconhecimento, modelo e síntese. Estados permitem informar se o sistema está ouvindo, reconhecendo, pensando, falando, cancelado ou em erro.

13.2 Reconhecimento de fala

O backend Whisper por processo exige executável e modelo compatíveis. Variáveis como `WHISPER_CLI` e `WHISPER_MODEL` evitam caminhos fixos. Teste primeiro um arquivo WAV conhecido; depois o microfone; por fim a integração.

Reconhecimento contínuo pode dividir áudio em blocos. No snapshot, o processamento continua síncrono, por isso a GUI deve usar worker apropriada. O operador precisa de indicador visível quando o microfone está ativo.

13.3 Voz não é autorização

Ruído ou reconhecimento incorreto pode transformar “não avance” em “avance”. Ações físicas obtidas por voz exigem confirmação por interface ou frase de confirmação com desafio claro. Palavras de parada devem ter tratamento local e prioridade, mas não substituem o botão de emergência.

Uma sequência recomendada é:

1. usuário: “vá um pouco para frente”;
2. STT produz texto;
3. agente propõe `MOVEFWD 90 500`;
4. aplicação apresenta a interpretação;
5. usuário confirma;
6. validação e firmware executam;
7. voz informa o resultado.

13.4 Síntese tradicional e clonagem

Para feedback operacional, TTS tradicional é suficiente e reduz dependências. Clonagem de voz é opcional. `TAIVoiceClone` exige consentimento por padrão, identificação do titular e áudio de referência autorizado. Não clone voz de terceiros sem autorização explícita.

O arquivo sintetizado só deve ser anunciado depois de existir, possuir tamanho e cabeçalho WAV válidos. Cancelamento deve alcançar STT, LLM e síntese.

13.5 Privacidade

Não grave microfone continuamente por padrão. Informe captura, retenção e destino. Quando STT ou LLM remoto for usado, áudio ou texto pode sair do dispositivo. Para demonstração em sala, evite armazenar voz dos participantes.

13.6 Critérios de aceitação

- Captura e reprodução funcionam isoladamente.
- STT acerta um conjunto de frases de teste em ambiente real.
- Cancelamento interrompe todas as etapas.
- Ação física sempre passa pela mesma validação do texto digitado.
- Consentimento de clonagem é verificável.
- Ausência de microfone ou modelo gera erro claro e não trava o robô.

14. Visão computacional

14.1 Correção histórica

O arquivo histórico `face.py` importa `VPython/visual` e desenha uma face animada. Ele não implementa reconhecimento facial com OpenCV e não deve ser apresentado como tal. O Motion realiza captura/detecção de movimento conforme configuração, mas também não equivale automaticamente a reconhecimento semântico.

14.2 Componentes atuais

Visão computacional transforma pixels em medidas ou observações. Três tarefas aparecem com frequência:

Tarefa	Pergunta respondida	Exemplo no Robotinics
Classificação	o que predomina na imagem?	"corredor" ou "bancada"
Detecção	quais objetos existem e onde estão?	peessoa em uma caixa delimitadora
Segmentação	quais pixels pertencem a cada região?	área livre do piso

Filtros tradicionais, como conversão para cinza, limiar e detecção de bordas, são rápidos e explicáveis. Redes neurais ajudam em variações maiores de forma e iluminação, mas exigem modelo, dados, memória e validação. A versão atual do módulo DNN do [OpenCV](#) oferece exemplos de classificação, detecção e segmentação; escolha a tarefa mínima necessária.

Para a câmera principal, a montagem de referência usa Camera Module 3 com Picamera2. A AI Camera com IMX500 é uma alternativa para inferência local de baixa latência, mas exige modelos compatíveis e instalação própria. Uma webcam USB também funciona quando driver, identificação persistente, latência e consumo forem validados.

`TAIOpenCV` possui dois backends:

Backend	Uso recomendado	Limitação
<code>ocvPythonProcess</code>	processamento real de imagem por worker Python	depende de Python, OpenCV e NumPy
<code>ocvNativeDLL</code>	detecção e carregamento do runtime	processamento nativo completo ainda é parcial/experimental

Na versão congelada, filtros reais incluem gray, blur, Canny, threshold e resize no backend Python. O demo combinado menciona tracking simulado em partes; portanto, ele não deve ser usado como prova de reconhecimento real.

Para detecção de faces, objetos ou pose, use componentes e samples dedicados, confirme backend real e valide com imagens de teste. `TAIHumanPoseDetector` requer plataforma 64 bits no estado documentado.

14.3 Pipeline do Robotinics

O pipeline recomendado é:

1. capturar frame com timestamp;
2. validar dimensões e formato;
3. reduzir resolução quando apropriado;
4. executar filtro ou detector especializado;
5. converter resultado em observação estruturada;
6. aplicar limiar e estabilidade temporal;
7. fornecer a observação ao planejador;
8. manter segurança de proximidade independente.

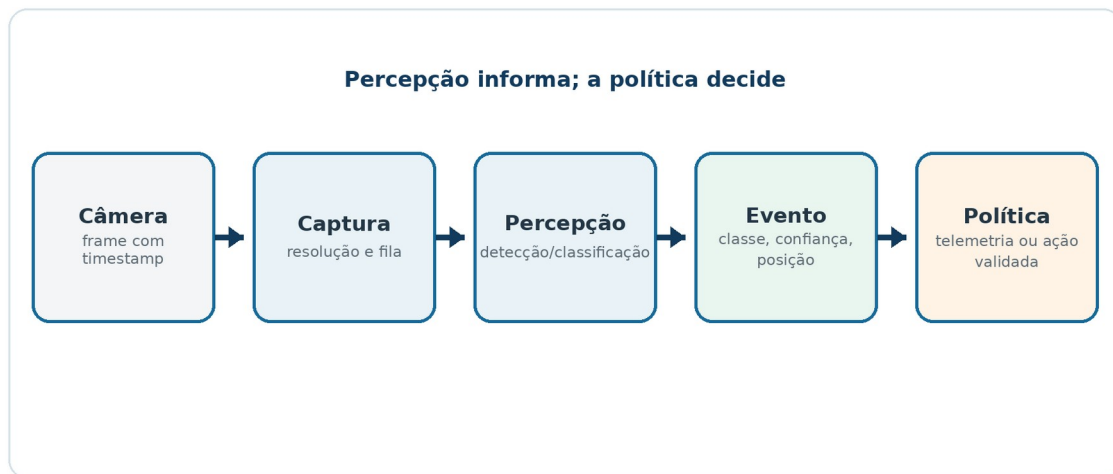


Figura 20 - Pipeline de visão separado do controle de movimento.

Fonte: Elaboração própria.

Não envie a imagem inteira ao LLM quando uma detecção local simples responde à pergunta. Um detector pode produzir `PERSON confidence=0.91 x=...`; o agente recebe essa observação e decide se deve explicar, perguntar ou propor ação.

Primeiro experimento com Picamera2 e OpenCV

Faça este teste antes de instalar detector ou LLM. Ele confirma câmera, formato e OpenCV e grava uma evidência sem controlar o robô.

PYTHON

```

from picamera2 import Picamera2
import cv2
import time

camera = Picamera2()
config = camera.create_still_configuration(
    main={"size": (1280, 720), "format": "RGB888"}
)
camera.configure(config)
camera.start()
time.sleep(1)

frame = camera.capture_array()
gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
edges = cv2.Canny(gray, 80, 160)
  
```

```
cv2.imwrite("robotinics_frame.jpg", frame)
cv2.imwrite("robotinics_edges.png", edges)
camera.stop()
```

1. Execute o script com o robô desarmado.
2. Confirme que os dois arquivos possuem 1280×720 pixels.
3. Verifique orientação, foco e enquadramento da câmera instalada na cabeça.
4. Repita sob luz forte, fraca e lateral.
5. Registre tempo de captura, temperatura e consumo de memória.
6. Somente depois acrescente detecção e transforme o resultado em evento estruturado.

Contrato de uma observação

```
JSON
{
  "schema": "robotinics.vision.v1",
  "timestamp": "2026-08-12T14:30:00Z",
  "class": "person",
  "confidence": 0.91,
  "bbox": [0.18, 0.12, 0.43, 0.82],
  "frame_age_ms": 74
}
```

Coordenadas normalizadas evitam dependência de resolução. `frame_age_ms` permite rejeitar percepção antiga. Classe e confiança não são ordens de movimento.

14.4 Desempenho no Raspberry

Meça FPS, latência, CPU, RAM, temperatura e consumo. Um modelo que funciona em desktop pode causar throttling no Raspberry. Ajuste resolução, intervalo de análise e modelo. Separe a captura do processamento para não acumular frames antigos; quando atrasado, descarte e use o frame mais recente.

14.5 Segurança e ética

Detecção de rosto não significa identificação confiável. Reconhecimento de pessoas envolve consentimento, viés, proteção de dados e risco de falsa identificação. Para o projeto educacional, prefira detecção de presença, cor, marcador ou objeto sem identidade.

Visão não deve ser única fonte de parada. Iluminação, oclusão e movimento causam falhas. Sensores locais e limites de velocidade continuam ativos.

14.6 Testes

- imagem válida, corrompida e ausente;
- câmera desconectada durante execução;
- variação de luz e fundo;
- objeto parcialmente oculto;
- frame atrasado;
- detector indisponível;

- alta carga de CPU;
- confirmação de que falha de visão reduz capacidade e não libera movimento.

15. Memória e RAG

15.1 Uso no robô

RAG permite responder com base em documentos do projeto sem treinar novamente o modelo. O Robotinics pode indexar:

- manual da Rev. 3;
- catálogo de componentes e fichas técnicas autorizadas;
- mapa de pinos;
- registros de manutenção;
- procedimentos de teste;
- histórico de falhas resumido;
- versão do protocolo.

O objetivo é apoiar diagnóstico e operação, não transformar texto recuperado em ação automática.

15.2 TAIRAG

TAIRAG integra `TAIGraphMap` e `TCHATGPT`. O fluxo mínimo é associar componentes, adicionar arquivos/pastas, construir índice, recuperar contexto e perguntar. A versão atual oferece modos grafo, vetor, BM25 e híbrido, orçamento de tokens e reranking opcional.

PASCAL

```
RAG.GraphMap := GraphMap;
RAG.ChatGPT := ChatGPT;
Agent.RAG := RAG;
Agent.ChatGPT := ChatGPT;

RAG.AddFolder('/opt/robotinics/docs');
RAG.BuildIndex;
```

Use filtros de extensão e diretório. Não indexe segredos, chaves, logs brutos ou dados pessoais sem necessidade.

15.3 Separação entre conhecimento e autoridade

Um documento pode dizer “para testar, envie FRENTE”. Esse texto não autoriza a ação. O RAG fornece contexto para resposta; apenas o catálogo do dispositivo e a política de segurança definem ações permitidas. Conteúdo recuperado é tratado como dado não confiável e não pode sobrescrever as instruções do agente.

15.4 Fontes e rastreabilidade

A resposta deve apresentar arquivos ou trechos usados. Armazene hash, versão e data do documento. Se duas revisões divergem, prefira a vinculada à versão do robô e informe o conflito.

Uma resposta útil é: “Segundo o mapa de pinos da revisão 3.0, o trigger frontal está no pino X; confirme a placa instalada.” Uma resposta ruim é afirmar certeza sem fonte ou misturar pinos de revisões diferentes.

15.5 Fatiamento

Chunks devem preservar unidade semântica: procedimento completo, definição com fórmula ou tabela com cabeçalho. Tamanho fixo é um ponto de partida, não solução universal. Use sobreposição moderada e metadados de capítulo, componente e versão.

Avalie com perguntas reais e conjunto de respostas esperadas. Meça recuperação da fonte correta antes de culpar o LLM. Um modelo pequeno pode responder bem se o contexto recuperado for curto e preciso.

15.6 Memória operacional

Separe:

- **estado atual:** telemetria volátil, nunca obtida de conversa;
- **memória de sessão:** diálogo e ações recentes;
- **memória de manutenção:** eventos confirmados e versões;
- **base documental:** conteúdo indexado e versionado.

Ao reiniciar, o estado físico começa desconhecido/desarmado, mesmo que a memória diga que o robô estava armado.

15.7 Avaliação

Crie um conjunto de perguntas:

- Qual a tensão nominal do pack 3S?
- Qual comando sempre deve parar o robô?
- Onde está o arquivo do firmware?
- O backend nativo do OpenCV processa imagens de forma completa?
- Qual revisão do mapa de pinos está instalada?

Registre fonte recuperada, resposta, acerto, latência e tokens. Inclua perguntas sem resposta; o sistema deve admitir ausência de evidência.

16. Comportamentos inteligentes e limites

16.1 Modos de operação

Modo	IA	Atuadores	Uso
Manual	opcional para explicação	operador envia comandos	diagnóstico inicial
Assistido	interpreta intenção	confirmação por ação	uso recomendado
Supervisionado	propõe sequência curta	limites e confirmação da sequência	tarefas validadas
Simulação	decide sobre modelo virtual	sem saída física	desenvolvimento e teste

Autonomia física sem supervisão não é objetivo inicial da Rev. 3. Ela só deve ser considerada depois que testes, localização, obstáculos, energia e recuperação de falhas estiverem validados.

16.2 Habilidades apropriadas

Primeiras habilidades:

- explicar estado e falha;
- descobrir e listar comandos;
- consultar sensores;
- propor movimento curto com confirmação;
- guiar checklist de manutenção;
- responder perguntas usando RAG;
- reconhecer frase e falar resposta;
- descrever observação de visão já estruturada.

Evite inicialmente:

- navegação prolongada sem mapa/localização;
- manipulação de objetos frágeis ou pessoas;
- execução de shell;
- alteração automática de firmware;
- uso irrestrito de internet;
- cadeia longa de ações sem confirmação ou feedback.

16.3 Planejar, executar e verificar

Uma tarefa é dividida em passos curtos. Antes de cada passo, valide estado. Depois, leia telemetria e compare resultado. Se o efeito não for confirmado, pare; não repita indefinidamente.

Exemplo “aproxime-se 30 cm”:

1. verificar sensor e estado;
2. propor deslocamento inicial curto;
3. confirmar;
4. executar por tempo limitado;
5. medir novamente;
6. recalcular com limite de iterações;
7. parar e informar resultado.

Sem odometria calibrada e sensor confiável, o sistema deve dizer que não consegue garantir 30 cm.

16.4 Incerteza

O agente precisa distinguir fato, inferência e ausência de dados. Telemetria com timestamp antigo não é “estado atual”. Detecção com baixa confiança não é presença confirmada. Documento de outra revisão não é autoridade para a montagem atual.

Defina limiares e respostas: perguntar ao operador, repetir leitura, reduzir velocidade ou recusar. “Não sei” é comportamento seguro.

16.5 Observabilidade de IA

Registre versão do modelo, provedor, temperatura, hash do prompt, TracelID, latência e resposta estruturada. Para privacidade, não armazene todo o conteúdo por padrão. Em caso de falha, deve ser possível responder: qual modelo propôs, qual validador bloqueou ou permitiu e qual comando o firmware recebeu.

16.6 Critério de promoção

Uma habilidade sai de simulação para operação física quando:

- possui contrato e limites documentados;
- todos os comandos pertencem ao catálogo;
- testes normais e adversariais passam;
- existe parada independente;
- falhas de rede/modelo são seguras;
- a interface informa claramente intenção e confirmação;
- resultados foram repetidos em hardware real;
- a versão do código foi congelada.

Integração, validação e evolução

17. Sequência de construção

17.1 Etapa 0 - documentação e inventário

Antes de comprar ou ligar componentes:

1. baixe a versão congelada do projeto;
2. gere inventário dos arquivos necessários;
3. identifique os componentes realmente disponíveis;
4. confira fichas técnicas e tensões;
5. registre substituições;
6. prepare esquema de alimentação;
7. defina botão de emergência e procedimento de teste.

Não use fotografias como único esquema. Produza mapa de conexões com nomes, pinos, tensão e corrente. Uma substituição de motor ou servo pode alterar mecânica, energia e firmware ao mesmo tempo.

17.2 Etapa 1 - mecânica passiva

Monte base, rodas, suportes e estrutura sem bateria. Verifique alinhamento, folgas e acesso. Instale pesos simulando bateria e placas para avaliar centro de gravidade. Braços permanecem sem carga e sem braços de servo durante o primeiro ajuste.

Aceitação: a base rola livre, não tomba nas poses permitidas e não apresenta interferência de cabos ou peças.

17.3 Etapa 2 - distribuição elétrica

Monte fusíveis, chave, conversores e bornes, mas teste inicialmente com fonte de bancada limitada em corrente. Valide um trilho por vez com carga eletrônica ou carga conhecida. Meça polaridade antes de conectar placas.

Aceitação: tensões permanecem dentro da tolerância durante transientes; não há aquecimento inesperado; desligamento remove energia dos atuadores conforme projeto.

17.4 Etapa 3 - Arduino sem atuadores

Carregue firmware Rev. 3. Use terminal serial para testar `PING`, `MAN`, `STATUS?`, linhas inválidas, overflow e timeout. Simule sensores quando possível. Verifique que o boot sempre informa `DISARMED`.

Aceitação: catálogo corresponde ao firmware, parser não executa prefixos parciais e falhas não bloqueiam o loop.

17.5 Etapa 4 - tração suspensa

Eleve as rodas. Conecte driver e motores com limite de corrente. Teste uma direção por vez, depois STOP, duração e watchdog. Confirme que esquerda e direita correspondem à convenção. Inverta fios ou configuração de forma documentada; não espalhe correções de sentido em várias rotinas.

Aceitação: partida, rampa, direção, parada e timeout são previsíveis; corrente e temperatura permanecem abaixo dos limites definidos.

17.6 Etapa 5 - servos e sensores

Conecte um servo por vez. Calibre zero e limites sem braço; depois instale elo sem carga e meça corrente. Sensores são validados individualmente, inclusive timeout e valores fora de faixa.

Aceitação: nenhuma articulação colide; perda de sensor relevante reduz capacidade; fonte não reinicia lógica durante movimento.

17.7 Etapa 6 - Raspberry e gateway

Instale o sistema, crie usuário de serviço e valide serial. O gateway consulta estado e registra telemetria, mas ainda não controla movimento por rede. Teste reinício, desconexão e permissões.

Aceitação: Raspberry pode reiniciar sem acionar atuadores; serviço recupera comunicação e informa versão.

17.8 Etapa 7 - TCHATGPT em simulação

Configure LLM e agente com `SimulationMode=True` ou sem a serial física associada. Use catálogo de teste e execute perguntas normais e adversariais. Verifique JSON, limites e confirmação.

Aceitação: o modelo não consegue ultrapassar catálogo e política; a interface permanece responsiva; credenciais não são persistidas em texto claro.

17.9 Etapa 8 - agente com hardware

Conecte a serial real com rodas suspensas. Descubra `MAN`, consulte estado e execute apenas `STOP`, `PING` e sensores. Depois autorize movimentos curtos. Monitore fisicamente e mantenha emergência ao alcance.

Aceitação: ação proposta, confirmação, comando, telemetria e log apresentam o mesmo `request_id`; qualquer divergência interrompe o teste.

17.10 Etapa 9 - voz, visão e RAG

Adicione um recurso por vez. Voz primeiro produz texto sem ação; visão primeiro produz observação; RAG primeiro responde com fontes. Somente depois cada recurso pode alimentar o agente, passando pelas mesmas barreiras.

Aceitação: falha do recurso adicional não altera a segurança básica; o operador sabe quando câmera/microfone estão ativos.

17.11 Primeira operação no piso

Use área plana, livre e demarcada. Comece com baixa velocidade e distância curta. Registre vídeo, telemetria, corrente e temperatura. Aumente gradualmente. Rampas e braços são fases posteriores, nunca parte do primeiro teste integrado.

18. Plano de testes e comissionamento

18.1 Pirâmide de testes



Figura 21 - Pirâmide de validação: muitos testes baratos sustentam poucos ensaios completos.
Fonte: Elaboração própria.

Testes começam em funções e módulos, avançam para integração e terminam no robô completo. Quanto mais físico e integrado, mais caro e perigoso é descobrir defeito. Parser, cálculo e política devem ser exercitados sem hardware.

18.2 Matriz mínima

ID	Ensaio	Condição	Resultado esperado
PWR-01	Polaridade	sem placas	todas as saídas corretas
PWR-02	Partida de servo	pior carga validada	lógica não reinicia
PWR-03	Subtensão	fonte reduzida controladamente	atuadores param e falha é registrada
FW-01	Boot	qualquer reset	DISARMED, motores desligados
FW-02	Linha longa	> buffer	erro, nenhuma execução
FW-03	Heartbeat perdido	durante movimento	STOP dentro do limite
FW-04	Comando desconhecido	linha válida	recusado
AG-01	Comando inventado	resposta do LLM	bloqueado pelo catálogo
AG-02	Parâmetro fora da faixa	PWM 999	bloqueado pelo validador
AG-03	Sem confirmação	movimento	não enviado
NET-01	Rede perdida	durante operação	segurança local preservada
VIS-01	Câmera ausente	início e runtime	erro claro, sem liberar movimento
VOI-01	Frase ambígua	ruído	pergunta/recusa, sem ação

ID	Ensaio	Condição	Resultado esperado
RAG-01	Documento conflitante	revisão antiga	conflito informado

18.3 Medições elétricas

Registre em cada modo:

Modo	V bateria	I bateria	V lógica mínima	I servo pico	Temperatura
Repouso					
Rodas suspensas					
Partida no piso					
Giro					
Braço, pior pose validada					

Use instrumento e método seguros. Uma medida isolada não representa pico rápido; osciloscópio ou registrador pode ser necessário para queda de tensão.

18.4 Medições mecânicas

- massa total e distribuição;
- diâmetro real das rodas;
- corrente e velocidade em piso plano;
- inclinação máxima validada;
- distância de parada por velocidade;
- folga de articulação;
- repetibilidade de posição;
- temperatura de motores e servos;
- estabilidade com braços em posições extremas.

18.5 Avaliação da IA

Crie conjunto fixo de frases em português, incluindo variações e erros. Classifique:

- resposta sem ação;
- consulta de sensor;
- ação válida;
- ação ambígua;
- ação proibida;
- ataque de prompt;
- pedido impossível;

- perda de contexto.

Para cada caso, registre JSON proposto, decisão do validador, necessidade de confirmação e resposta final. Trocar modelo exige repetir o conjunto; modelos com nomes semelhantes podem se comportar de forma diferente.

18.6 Injeção de falhas

Falhas devem ser provocadas de modo controlado:

- remover cabo serial;
- desligar servidor LLM;
- atrasar resposta;
- enviar bytes corrompidos;
- bloquear sensor;
- simular corrente alta por entrada de teste;
- reiniciar processo;
- encher espaço de log em ambiente de teste;
- fornecer documento RAG com instrução maliciosa.

O objetivo é verificar saída segura e diagnóstico, não danificar o equipamento.

18.7 Registro de ensaio

Cada ensaio contém:

TEXT
 ID e título:
 Data e responsável:
 Versão mecânica:
 Firmware e commit:
 Aplicação/TCHATGPT e commit:
 Hardware e configuração:
 Pré-condições:
 Passos:
 Resultado esperado:
 Resultado observado:
 Medições e evidências:
 Conclusão: PASS / FAIL / BLOQUEADO
 Anomalia vinculada:

18.8 Critério de comissionamento

O robô pode operar supervisionado quando todos os testes críticos passam, não há falha aberta de segurança e a configuração testada corresponde à montada. Uma demonstração bem-sucedida não substitui o conjunto de ensaios.

19. Manutenção e evolução

19.1 Inspeção periódica

Antes de cada sessão, verifique bateria, conectores, fusíveis, rodas, parafusos, cabos e emergência.

Depois de impactos ou transporte, repita inspeção mecânica e teste suspenso.

Periodicamente:

- confira capacidade e equilíbrio do pack com procedimento apropriado;
- limpe e inspecione motores e articulações;
- procure aquecimento e escurecimento em conectores;
- revise logs de subtenção, sobrecorrente e watchdog;
- faça backup de configuração e índice RAG;
- aplique atualização apenas com plano de rollback;
- repita testes afetados.

19.2 Compatibilidade

Mantenha uma matriz de sistema operacional, arquitetura, Lazarus, FPC, pacotes e runtime. “Compila no Windows” não prova Linux ou ARM. O CI deve construir samples por pacote e um conjunto Robotinics específico. Testes em hardware ficam separados dos testes de compilação.

19.3 Repositório reproduzível

A Rev. 3 recomenda:

1. tag do Robotinics correspondente ao livro;
2. diretório `docs/rev3` com fonte editável do livro;
3. firmware Rev. 3 separado do legado;
4. sample Lazarus `robotinics_ai_controller`;
5. esquemas e BOM versionados;
6. script de instalação e verificação;
7. CI para firmware e Lazarus;
8. releases com checksum;
9. política de vulnerabilidades e contribuição.

Arquivos compilados, objetos e caches não devem substituir fontes. Ferramentas históricas removidas podem ficar em tag `Legacy`, com aviso de segurança.

19.4 Roadmap técnico

Prioridade 0 - segurança e reprodução

- validar pack, BMS, carregador, fusível e trilhos;
- implementar firmware Rev. 3 com watchdog;
- corrigir mapa de pinos e comandos;
- criar parada física e teste de falha;
- congelar BOM e versões.

Prioridade 1 - controlador inteligente

- adaptar `agent_serial_demo` ao Robotinics;
- integrar `TAIAgentSafety` e validador específico;
- criar modo simulação;
- adicionar telemetria e TraceID;
- testar Windows/Linux x64.

Prioridade 2 - Raspberry ARM64

- criar CI ou build reproduzível ARM64;
- validar `openai_core`, serial e agente;
- validar voz e visão separadamente;
- medir desempenho e temperatura;
- promover componentes somente com evidência.

Prioridade 3 - percepção e conhecimento

- RAG com documentos versionados;
- voz com consentimento e confirmação;
- visão com backend real identificado;
- datasets de avaliação;
- observabilidade e regressão.

19.5 Publicação

Antes de publicar:

- definir licença do texto e das figuras;
- manter a licença GPLv3 e avisos aplicáveis ao TCHATGPT;
- auditar origem de imagens e modelos de terceiros;
- remover senhas, tokens e dados pessoais;
- verificar links e commits;

- revisar tecnicamente bateria, elétrica e mecânica;
- gerar PDF com bookmarks, links e texto copiável;
- oferecer DOCX ou fonte equivalente para futuras revisões.

O repositório Robotinics não apresenta licença formal no snapshot examinado. A alegação de projeto aberto deve ser acompanhada por um arquivo de licença escolhido pelo autor. Esta edição não escolhe a licença em nome dele.

Apêndice A - Lista de materiais orientativa

Esta lista organiza categorias. Modelo e quantidade finais dependem da montagem e devem constar na BOM versionada.

Subsistema	Itens	Verificação
Estrutura	peças STL/CAD, parafusos, insertos, suportes	material, revisão, carga e folga
Tração	motores com redução, rodas, driver	tensão, corrente de stall, torque contínuo
Controle	Arduino Mega ou placa compatível	pinos, memória, interfaces
Distribuição	shield Mega com conectores carenados, carcaças e terminais compatíveis	passo, trava, pinagem, corrente e separação de Vservo
Computação	Raspberry Pi 5, fonte 5 V/5 A, resfriamento e armazenamento	ARM64, alimentação, temperatura e identificação persistente
Energia	pack 3S, BMS, carregador, fusível, chave	química, corrente, certificação e montagem
Conversão	trilho de tração, servos e lógica	tensão, corrente, ripple e refrigeração
Servos	atuadores e suportes	torque calculado, limites e corrente
Sensores	ultrassom, tensão, corrente, IMU/gás conforme uso	nível lógico, faixa, calibração
Interface	câmera, microfone, alto-falante	driver, consentimento, consumo
Segurança	emergência, proteções, fixação	acesso, teste e falha segura
Fiação	fios, bornes, conectores, etiquetas	corrente, queda, polarização e flexão

Não compre pela BOM histórica sem conferir disponibilidade e especificação. Clones com o mesmo nome comercial podem diferir.

Apêndice B - Protocolo de comandos

B.1 Regras

- ASCII/UTF-8 restrito para comandos; uma linha por mensagem.
- Terminação LF ou CRLF.
- Máximo de 127 bytes de conteúdo por linha na configuração de referência.
- Primeiro token identifica exatamente o comando.
- Números decimais sem unidade somente quando o contrato fixa unidade.
- `STOP`, `DISARM`, `PING`, `STATUS?` e `MAN` não dependem de IA.
- Movimento exige estado `ARMED`.
- Respostas incluem `OK`, `ERR`, `STATE` ou `EVENT`.

B.2 Catálogo mínimo

Comando	Parâmetros	Estado	Efeito
PING	nenhum	qualquer	responde OK PING
MAN	nenhum	qualquer	publica catálogo
STATUS?	nenhum	qualquer	retorna estado e falhas
ARM	conforme política	DISARMED	executa autoteste e arma
DISARM	nenhum	qualquer	desabilita atuadores
STOP	nenhum	qualquer	interrompe movimento
MOVEFWD	PWM, ms	ARMED	avanço limitado
MOVEBACK	PWM, ms	ARMED	ré limitada
TURNLEFT	PWM, ms	ARMED	giro limitado
TURNRIGHT	PWM, ms	ARMED	giro limitado
RANGE?	opcional sensor	qualquer	distância ou timeout
BATTERY?	nenhum	qualquer	tensão e estado
SERVO	eixo, ângulo, velocidade	ARMED	pose dentro dos limites

B.3 Aliases legados

Legado	Rev. 3	Observação
FRENTE	MOVEFWD	parâmetros padrão conservadores
RE	MOVEBACK	parâmetros padrão conservadores
GESQ	TURNLEFT	duração limitada
GDIR	TURNRIGHT	duração limitada
PARA	STOP	PARAR pode ser aceito apenas como alias documentado

Aliases não contornam estado, catálogo ou limites.

Apêndice C - Matriz de correções da edição anterior

Local aproximado	Problema identificado	Correção adotada na Rev. 3
pp. 32-45	força em N comparada diretamente a torque em N·m	fórmulas separadas e conversão por raio
pp. 38-39	massa, peso, kgf, N e inclinação misturados	$m \times g \times \sin(\theta)$ e exemplo dimensional
pp. 42-45	torque de braço em $\text{kgf} \cdot \text{cm}^2$ e massas inconsistentes	soma de momentos em N·m, centros de massa e margem
pp. 49-50	$0,02 \text{ A} = 200 \text{ mA}$	$0,02 \text{ A} = 20 \text{ mA}$
pp. 53-56	potência aparente/reativa aplicada ao pack DC	cálculo por W, Wh e eficiência de conversão
pp. 58-59	capacidade de células em série somada	3S mantém 5,2 Ah; soma apenas tensão
p. 96	$20,75 \times 10^{-3} \text{ F}$ convertido em 20,75 μF	valor correto: 20.750 μF
pp. 97-113	carga de lítio baseada em corrente incorreta e permanência indefinida	carregador CC/CV 3S, BMS, balanceamento e ficha técnica
pp. 108-113	buck ajustado para 12,6 V com entrada de 12 V	buck não eleva tensão; arquitetura redesenhada
p. 106	NA/NF invertidos	NO aberto em repouso; NC fechado em repouso
p. 113	amperímetro ligado em paralelo na saída	corrente medida em série; aviso de curto
pp. 135-146	correntes de trilhos somadas e fonte única subdimensionada	orçamento por potência e trilhos separados
pp. 140-144	L298N descrito sem PWM/como isolamento	PWM nos enables, sem isolamento, perdas explicitadas
pp. 150-155	HC-SR04 indicado como 40 Hz	aproximadamente 40 kHz
exemplo ultrassom	echoPin/trigPin indefinidos e mesma amostra repetida	pinos definidos, timeout e amostras distintas
pp. 187-193	servo associado apenas a pinos PWM	biblioteca usa temporizadores e pinos digitais compatíveis
p. 196	$5/330 = 0,15 \text{ A}$	$5/330 \approx 0,015 \text{ A}$; fórmula usa queda do LED
p. 216	exemplo MySQL com aspas tipográficas e erros de sintaxe	exemplos compiláveis e consultas parametrizadas
pp. 218-220	servidores C/C++ com buffers, formato e shell inseguros	retirados da arquitetura operacional; gateway validado
pp. 221-223	Motion confundido com visão semântica	captura/movimento separados de detecção/classificação
pp. 252-255	OpenCV 2.4 e instalação antiga	referência à documentação atual e backends TCHATGPT
face.py	animação VPython apresentada como reconhecimento	classificado corretamente; usar detector real
pp. 256-267	PocketSphinx antigo como única rota	Whisper/STT modular, mantendo Sphinx apenas como legado
pp. 268-275	FANN e exemplos ausentes/incompletos	componentes ML atuais ou documentação explicitamente histórica
pp. 280-297	PHP5, root MySQL, phpMyAdmin exposto, chmod 777	privilegio mínimo, serviço local, autenticação e permissões
firmware	String, delays, buffer compartilhado e sem timeout	parser fixo, módulos, watchdog e limites
firmware	macros GPS inválidas e risco de overflow/bloqueio	definições corretas, buffer limitado e timeout
firmware	ULTRA antes de ULTRA1/2	comparação exata do primeiro token
firmware	substring(5) para prefixo SERIAL :	parser por token e tamanho real

Local aproximado	Problema identificado	Correção adotada na Rev. 3
ferramentas	portas 7091/8091 e PARAR/PARA divergentes	contrato central e aliases documentados
repositório	ferramentas citadas removidas do branch atual	snapshot e histórico declarados; novo sample proposto
editorial	capítulo 6.2 ausente, links sem clique, sem bookmarks	numeração contínua, navegação e links
PDF	afirmação de abertura sem licença formal	decisão de licença exigida antes da publicação

Apêndice D - Fórmulas de referência

Grandeza	Fórmula	Unidade
Peso	$F = m \times g$	N
Força de atrie	$F = m \times g \times \sin(\theta)$	N
Rolamento	$F = C_{rr} \times m \times g \times \cos(\theta)$	N
Aceração	$F = m \times a$	N
Torque	$T = F \times r$	N·m
Torque de braço	$T = \Sigma(m_i \times g \times d_i)$	N·m
Relação de redução	$i = Z_{saída}/Z_{motora}$	adimensional
Rotação após redução	$n_{saída} \approx n_{motor}/i$	rpm
Torque após redução	$T_{saída} \approx T_{motor} \times i \times \eta$	N·m
Potência DC	$P = V \times I$	W
Energia	$E = P \times t$ ou $V \times Ah$	Wh
Resistor de LED	$R = (V_{fonte} - V_{LED}) / I$	Ω
Potência no resistor	$P = I^2 \times R$	W
Capacitor de ripple	$C \approx I / (f \times \Delta V)$	F
Autonomia	$t \approx E_{utilizável}/P_{média}$	h
Distância ultrassônica	$d = v_{som} \times tempo/2$	m

Use unidades coerentes. Converta milímetros para metros e miliampères para ampères antes de calcular. Declare hipóteses e arredondamento.

Apêndice E - Matriz de plataforma e dependências

Camada	Windows x64	Linux x64	Raspberry ARM64	Observação
openai_core	suportado	suportado	provável/experimental	validar HTTPS e runtime
serial/input	suportado por samples	suportado por samples	provável/experimental	permissões e dispositivo
agente serial	compila no relatório	compila conforme ambiente	experimental	testar hardware e GUI
voz	depende de executáveis/modelos	depende de executáveis/modelos	experimental	custo de CPU/RAM
OpenCV Python	depende de Python/OpenCV	depende de Python/OpenCV	experimental	usar TAIPythonRuntime
OpenCV nativo	carregamento parcial	carregamento parcial	experimental	processamento nativo incompleto
RAG	compila por sample	esperado	experimental	medir memória e índice

Não promova “provável” para “suportado” sem build e teste repetível.

Apêndice F - Referências

Projetos

1. Marcelo Maurin Martins. **Robotinics**. [Repositório do Robotinics](#). Base editorial 39c1b4610f1b51883a2d9fcdccba2c672dafad3.
2. Marcelo Maurin Martins. **TCHATGPT - AI Component Suite for Lazarus / Free Pascal**. [Repositório do TCHATGPT](#). Snapshot editorial 15d5e1a7780088701716896cfe9fb3afe0e7b71a.
3. TCHATGPT. [Compatibilidade por plataforma](#).
4. TCHATGPT. [Agent Serial Demo](#).
5. TCHATGPT. [Arquitetura de LLM e providers](#).

Documentação técnica oficial

6. Raspberry Pi Ltd. [Getting started - Raspberry Pi Documentation](#).
7. Arduino. [Arduino IDE 2 - Getting Started](#).
8. OpenCV. [Installation in Linux](#).
9. Texas Instruments. [Cell balancing buys extra run time and battery life](#).
10. Texas Instruments. [Precise Constant Current Regulation Helps Advance Fast-charging](#).
11. Fluke. [Digital multimeter safety and measurement guidance](#).
12. Omron. [Explanation of relay terms](#).
13. Raspberry Pi Ltd. [Raspberry Pi 5](#).
14. Raspberry Pi Ltd. [Compute Module 5](#).
15. Raspberry Pi Ltd. [Camera hardware](#).
16. Raspberry Pi Ltd. [Camera software and Picamera2](#).
17. Raspberry Pi Ltd. [Raspberry Pi AI Camera](#).
18. OpenCV. [Deep neural network tutorials](#).
19. ggml-org. [llama.cpp](#).
20. Molex. [KK connectors](#).
21. Achei Componentes. [Connfly DS1069-4MVW, quatro vias, passo de 2,5 mm](#).
22. Arduino. [Arduino Mega 2560](#).

Apêndice G - Glossário

Termo	Definição
ADC	conversor analógico-digital
BMS	sistema de gerenciamento/proteção de bateria
CC/CV	carga por corrente constante e tensão constante
CI	integração contínua; compilação/teste automatizado
Firmware	software executado no microcontrolador
GPIO	entrada/saída de propósito geral
Heartbeat	mensagem periódica de presença/saúde
LLM	modelo de linguagem de grande porte
Conector carenado	conector com paredes que guiam e protegem o encaixe
Conector polarizado	conector cuja geometria reduz a possibilidade de inversão
Crimpagem	união mecânica e elétrica de fio e terminal por deformação controlada
PWM	modulação por largura de pulso
RAG	geração aumentada por recuperação de documentos
Stall	condição de eixo travado do motor
STT	conversão de fala para texto
TTS	conversão de texto para fala
Watchdog	temporizador que detecta travamento ou ausência de atualização

Encerramento

O Robotinics continua sendo mais valioso como plataforma aberta de aprendizagem do que como produto fechado. A terceira edição não apaga a história: ela transforma os erros, limitações e mudanças de tecnologia em parte do ensino.

O princípio central é simples: inteligência artificial amplia a capacidade de conversar, perceber e planejar, mas a segurança do robô permanece determinística, local, mensurável e testável. Com essa divisão, o projeto pode evoluir sem confundir criatividade com autorização.

O próximo passo pertence à comunidade: reproduzir, medir, registrar e melhorar, sempre preservando a origem e a capacidade de verificar cada decisão.